LLM-Based Eco-Solutions Patent Analyzer

FIT3164 Data Science Project 2

Final Project Report

MDS08

10874 words

Shrinithi Venkatesh (33512957)
Quality Assurance & Data Developer
sven0022@student.monash.edu

Steve Jevon Rahardjo (32153325)
Technical Lead
srah0021@student.monash.edu

Naveed Mahmud (33304521)
Software Developer
nmah0024@student.monash.edu

Renee Yeo Shu Ting (33518904)
Project Manager & UI Developer
rshu0014@student.monash.edu

Dr Ong Huey Fang
Project Supervisor
ong.hueyfang@monash.edu

**Word count excluding in-text citations, front cover page, table of contents, headings, figure or table labels, appendix and references

Table of Contents

1.0 Introduction *3*

2.0 Project Background *4*

2.1 Context and Motivation: AI for Sustainable Innovation **4**

2.2 TRIZ: A Structured Framework for Eco-Innovation **5**

2.3 Challenges in Manual Classification and the Case for LLMs **6**

2.4 The Role of Retrieval and Hybrid Systems **6**

2.5 Why Our Approach is Different **7**

3.0 Outcomes *8*

3.1 What Has Been Implemented **8**

3.2 Results Achieved **10**

3.3 How Are Requirements Met **13**

3.3.1 Mult-Label Classification System 14

3.3.2 Retrieval-Based Chatbot 15

3.3.3 Web Application 15

3.4 Justification of Decisions Made **16**

3.4.1 Decoder Model Selection 16

3.4.2 Separate Frontend-Backend Design 16

3.4.3 Chatbot Interface to Help User Inquiry 17

3.5 Discussion of Project Results **17**

3.6 Limitations of the Project **17**

3.6.1 Inexpertise in the TRIZ Framework 17

3.6.2 Choice of Document Processor: Selenium vs OCR 18

3.6.3 Dependence on Closed-Source LLM Model 18

3.7 Improvements and Future Works **18**

3.8. Critical Discussions on Outcomes **20**

4.0 Methodology *21*

4.1. Design **21**

4.1.1 High-Level Explanation of Application Flow 22

4.1.2 Deviation from Proposed Design 22

4.2 Tech Stack **23**

4.2.1 Hardware Requirements 23

4.2.2 Software Requirements 23

4.3 Design Implementation **25**

4.3.1 Document Preprocessing 25

4.3.2 Multi-Label Classification 26

4.3.3 Retrieval-Based Chatbot 28

4.3.4 Backend Design 29

4.3.5 Frontend Design 29
5.0 Deliverables *30*
5.1 Project Deliverables **30**
5.2 Software Deliverables **30**
5.2.1 Summary of Software Deliverables 30
5.2.2 Main Software Features 30
5.2.3 Software Quality Summary 35
5.2.4 Sample Source Code 37
6.0 Critical Discussion *37*
6.1 Project Overall Success **37**
6.2 Project Management **39**
6.2.1 Methodology 39
6.2.2 Schedule and Scope 39
6.2.3 Team Management and Collaboration 39
6.2.4 Risk Management 40
7.0 Conclusion *40*
8.0 Appendices *42*
9.0 References *55*

1.0 Introduction

With the rise of newly developed products and environmentally driven innovations since the 1990s, companies across various industries have found it increasingly challenging to navigate patent classification (Faria & Andersen, 2014). As the number of eco-solution patents continued to grow

over time, stakeholders, inventors, and patent examiners faced mounting challenges, burdened by the manual effort required to categorize these solutions. However, the traditional methods for classifying specific problem-solving approaches often fell short, resulting in researchers finding it challenging to achieve accurate and consistent classifications (Loh et al., 2005).

In this project, Team MDS08 developed a web-based application called EcoPatent Analyzer, leveraging large language model (LLM) technologies to classify patent content based on the 40-TRIZ Inventive Principles and provide structured analytical insights. Our primary goal was to automate the patent analysis process - extracting key information, mapping it to relevant TRIZ principles, and presenting the results within an accessible web-based interface. Specifically, we aimed to:

- Develop a web platform allowing users to upload a USPTO patent PDF and receive an immediate analysis of its key inventive principles based on the TRIZ methodology.
- Integrate a conversational AI chatbot to provide plain-English explanations for each classification, enabling users to query the patent contents interactively.
- Implement interactive visualizations to help users identify trends and compare inventive approaches across different patents at a glance.
- Ensure the final application is reliable, secure, and built upon a maintainable architecture for future expansion.

This report is structured to provide an overview of how the team developed the project and its outcomes. It begins with Section 2, which highlights the challenges in manual classification and how we retrieve our data using the RAG pipeline. Section 3 discusses the implementations, including multi-label classification, conversational chatbot, front-end interface, and document preprocessing. This section also evaluates how well the project met its functional and non-functional requirements. Section 4 describes the methodology applied throughout the project, including the system's design and development processes. Section 5 outlines the software deliverables, including the main software features and their usage. Additionally, we also discussed the handling of software qualities such as robustness and maintainability. Section 6 provides a critical evaluation of the project as a whole, assessing its success in achieving the intended goals and reflecting on the development approach and team collaboration. Lastly, Section 7 summarizes the overall content of this report, followed by Appendices and References to support the technical content and citations provided.

2.0 Project Background

2.1 Context and Motivation: AI for Sustainable Innovation

As the global climate crisis intensifies, industries are facing increasing pressure to develop sustainable solutions that reduce emissions, conserve resources, and promote circularity. This

demand for eco-innovation is particularly evident in sectors such as energy, transportation, and manufacturing. However, sustainable product development often presents significant contradictions; enhancing durability may lead to increased material usage, or improving performance may generate more waste. Eco-design is crucial in addressing these trade-offs by integrating sustainability into the early stages of product development.

A valuable source of technical inspiration for eco-innovation is patent literature, which encompasses millions of documented inventions. Green patents, especially those related to electric vehicles, renewable energy, and material substitution, provide a wealth of actionable knowledge for designing low-impact technologies (Associados, 2025; EXPLAINER: How Intellctual Property Rights Encourage Green Innovation, n.d). In the steel industry, structured patent searches have become an essential component of research and development strategies (IP Business Academy, 2025). However, manually classifying and analyzing patents remains labor-intensive, subjective, and challenging to scale.

To overcome this bottleneck, frameworks like TRIZ have been applied to map patents to problem-solving strategies that support sustainability (Russo & Spreafico, 2020). Despite these structured methods, the process still relies heavily on human expertise and interpretation. The lack of automation limits accessibility, delays knowledge extraction, and hinders the timely application of green design strategies across various domains.

This project aims to bridge the gap between sustainability and artificial intelligence by automating the TRIZ classification of patents using large language models (LLMs). This approach will enable scalable and systematic extraction of eco-innovation pathways that were previously hidden behind dense technical documentation.

2.2 TRIZ: A Structured Framework for Eco-Innovation

TRIZ (Theory of Inventive Problem Solving) is a systematic methodology that helps identify patterns of innovation. It was initially developed by Genrich Altshuller through the analysis of over 200,000 patents. TRIZ extracts common strategies to resolve design contradictions. Rather than relying on trial-and-error ideation, TRIZ allows us to categorize patents into patterns, helping us understand how a solution works and how inventive it truly is.

This framework is particularly powerful in the context of eco-design, where innovation must strike a balance between technical performance and sustainability goals. TRIZ helps navigate conflicting design parameters, such as reducing material usage without compromising strength, through tools like the contradiction matrix and 40 inventive principles.

However, applying TRIZ manually to patent databases can be a complex, subjective, and time-consuming process. As Russo and Spreafico (2020) explain, while TRIZ-based eco-guidelines provide targeted sustainability strategies, their real-world adoption is limited due to the manual expertise required. Automating TRIZ classification is essential for scaling its benefits, making

structured eco-innovation accessible to non-experts and applicable across large volumes of unstructured patent data.

2.3 Challenges in Manual Classification and the Case for LLMs

While TRIZ provides a powerful lens for structured innovation analysis, applying it manually to large patent corpora remains highly impractical. Traditional TRIZ assessments require a rare combination of deep domain knowledge, expert-level patent interpretation, and fluency in TRIZ principles (Russo & Spreafico, 2020). As a result, classification processes are slow, subjective, and poorly scalable, especially in sustainability domains where patents are complex, dense, and cross-disciplinary.

Manual workflows introduce further issues of reproducibility and cognitive fatigue. Even semi-automated systems based on TRIZ guideline matrices (Russo & Spreafico, 2020) depend heavily on expert judgment, making them unsuitable for real-time analysis or high-throughput eco-innovation pipelines.

To address these limitations, we leverage the capabilities of Large Language Models (LLMs) for TRIZ classification. LLMs can process long, technical documents, generalize across innovation domains, and follow structured instructions. Through prompt chaining (Wei et al., 2022), they can simulate logical step-by-step reasoning. This is further enhanced using reasoning trace injection techniques (Johnson, 2025b), which make the decision-making process transparent and verifiable.

Modern LLMs have shown the ability to respond coherently to domain-specific prompts, perform complex reasoning, and reduce inconsistency in outputs. These strengths align precisely with the requirements of TRIZ classification, where each label depends on subtle interplays between invention goals and technical constraints (Chiang et al., 2024; Venugopalan, 2025; Zhou et al., 2023).

Our system builds on this potential by deploying an LLM-driven classification pipeline that automates TRIZ mapping at scale, enabling consistent, explainable insights across diverse patent datasets in sustainability.

2.4 The Role of Retrieval and Hybrid Systems

Despite the impressive capabilities of Large Language Models (LLMs), they often generate outputs that lack factual grounding, which is particularly problematic in technical domains, such as patent analysis. In our system's chatbot module, this limitation is mitigated through a Retrieval-Augmented Generation (RAG) pipeline that anchors model responses in relevant, factual content extracted directly from patent documents.

To ensure precision and coverage, we implement a hybrid retrieval strategy combining both sparse and dense retrieval techniques. Specifically, we utilize BM42, an improved sparse retriever that

builds on the BM25 model by incorporating transformer-based attention scores to improve token-level relevance (Vasnetsov, 2024). Alongside this, we use OpenAI's Ada-002 to generate dense semantic embeddings that capture the contextual meaning of user queries and document chunks.

To further optimize this retrieval process, we integrate Reciprocal Rank Fusion (RRF), which merges ranked outputs from both retrieval modes by scoring and aggregating their positions. This fusion boosts the robustness of top-k results, improving recall without sacrificing specificity. Additionally, we employ binary quantization, a vector compression technique that converts 32-bit floating-point embeddings into compact binary formats. This results in up to 85× reduction in storage and significant speed-ups in vector search, making the pipeline scalable and efficient (Kasliwal, 2023b).

To assess the chatbot's effectiveness in accordance with its intended design, specifically in providing factually grounded responses based on the source of patent documents it loads, we conducted evaluations according to the **RAGAS paper (**Es et al., 2023**).** This approach ensures that the chatbot's answers are not only contextually appropriate but also verifiable against the source documents, which is critical for maintaining trust and reliability in patent-related inquiries. The RAGAS framework streamlines evaluation by using Large Language Models (LLMs) as evaluators. Instead of extensive human annotation, RAGAS takes the user's query, retrieved context (patent passages), and the chatbot's answer, then prompts an evaluator for LLM to assess specific quality dimensions. The metric we have used:

- Context Recall
  Measures how many of the relevant documents (or pieces of information) were successfully retrieved. It focuses on not missing important results. Higher recall means fewer relevant documents were left out.
- Faithfulness
  Assesses the degree to which the generated answer is factually supported by the given context, in our case, the retrieved patent passages.
- Noise Sensitivity
  Measures how often a system makes errors by providing incorrect responses when retrieving based on the provided ground truth label.
- LLM Context Precision Without Reference
  Measures the proportion of relevant chunks being retrieved compared to the result being produced by the system; an insufficient number of these means that the retriever is not efficient and is over-retrieving.

2.5 Why Our Approach is Different

Our project advances TRIZ-based patent analysis by transforming it from a heuristic-driven, manual process into a fully automated and explainable reasoning pipeline. While our previous FIT3163 literature review established the foundations of TRIZ (Sojka & Lepsik, 2020) and the

evolution of LLMs for NLP (Kamath et al., 2024), its focus was primarily on symbolic AI and rule-based systems. These traditional approaches, such as the text-mining techniques proposed by Tseng et al. (2007), faced significant limitations with the ambiguity of technical language. Our system moves beyond these static rules by implementing decoder-based LLM prompting, which enables a more dynamic and context-aware classification.

In this report, we expand that foundation by introducing an end-to-end pipeline that delivers transparent and verifiable classifications. Early computer-aided TRIZ tools, such as those developed by Cascini and Russo (2006), required considerable human expertise for interpretation, thereby hindering scalability. In contrast, our approach leverages multi-stage prompt chaining and reasoning trace injection (Johnson, 2025b) to automate the analytical process while making the LLM's logic transparent. This novel mechanism enhances consistency across complex patent claims and minimizes the subjective variability that plagues traditional TRIZ workflows.

Furthermore, while prior research by Senghor et al. (2023) and Albino et al. (2014) effectively highlighted TRIZ's potential for driving eco-innovation, it did not offer the scalable tooling necessary to analyze large patent corpora efficiently. Our solution directly addresses this gap by embedding LLMs into a pipeline tailored for sustainability contexts, enabling active innovation discovery rather than passive patent tagging.

Compared to the earlier FIT3163 prototype, this system also integrates a sophisticated chatbot layer powered by state-of-the-art hybrid retrieval methods (BM42 + Ada-002). This improves upon existing LLM-based retrieval work (Johnson, 2025b) by using quantization-accelerated RAG to provide fact-grounded semantic querying. Crucially, this retrieval module is decoupled from the classification system, a key design choice that resolves methodological limitations noted in the literature (Tseng et al., 2007; Russo & Spreafico, 2020) and ensures the interpretability of our core TRIZ analysis. By combining automation, interpretability, and domain adaptation, we provide a scalable architecture that enables researchers to identify design contradictions, discover inventive patterns, and apply TRIZ sustainably. In the sections that follow, we describe how this architecture delivers a cohesive, research-grade platform for large-scale green patent analysis.

3.0 Outcomes

3.1 What Has Been Implemented

(a) Patent Document Processing

The file processing module ensures that uploaded documents meet the application's scope and technical requirements, enabling stable backend operations and preventing unexpected processing errors. The document processing follows these key steps:

1. **File Validation:** The system checks whether the uploaded file is a valid PDF and verifies that it originates from the United States Patent and Trademark Office (USPTO) by parsing the file metadata and extracting the unique identifier.

2. **HTML Version Retrieval:** Once the appropriate identifier is obtained, the system retrieves the HTML version of the patent from the USPTO database. This version provides a structured format that includes essential sections such as Inventor, Publication Date, Summary, and Claims.
3. **HTML Content Parsing:** After retrieving the HTML document, the system parses its content to extract relevant sections, including the title, abstract, background, claims, and detailed description. This structured extraction isolates meaningful technical information required for downstream processing modules.

(b) Multi-Label Classification

This module is implemented to help our target user base understand ecological patent documents through the lens of the 40 TRIZ Inventive Principles, using multi-label classification powered by a decoder-based language model. It also generates a structured summary to provide context beyond the classification alone.

(c) Retrieval-Augmented Generation (RAG) Chatbot

This module is designed to provide users with an interactive interface for querying the ecological patent documents they uploaded to the system. It combines semantic and keyword-based retrieval methods through extensive, technical terminology-rich patent content. The retrieved information is then passed to a decoder-based language model, which generates structured responses that directly address the user's query. This process ensures that the chatbot's answers are grounded in factual content extracted from the source patent.

- **Execution**
  To support seamless user interaction, these modules are integrated into backend services that are triggered by frontend events such as file uploads, button clicks, and query inputs. This design ensures the application coordinates multiple services into a cohesive and responsive user flow.

(d) Web Application Development

A web application was developed with four main interfaces, each tailored to provide intuitive access to key patent analysis features.

Interface 1: Home Page

The Home Page introduces the system's purpose and provides navigation guidance to users. It presents an overview of the system's key features and serves as a welcoming entry point for everyone. The page is structured into four main sections, as shown in Figures 3.1.1 to 3.1.7, to support onboarding and usability.

It begins with a hero section (Figure 3.1.1), which includes a brief tagline and a prominent "Explore Now" button that redirects users directly to the Upload Page, facilitating quick access to the application's core functionality. This is followed by the "How It Works" section (Figure 3.1.2),

which outlines step-by-step instructions that guide users through the upload, classification, and interaction processes. Further down, an FAQ section (Figure 3.1.3) features interactive dropdowns addressing common user queries, including a brief explanation of TRIZ principles. The final section (Figure 3.1.4) showcases the system's technology stack, offering transparency into the tools and frameworks used to develop the application.

Interface 2: Insights Page

This page, shown in Figure 3.1.5, presents visual summaries and analytical insights derived from the vector database. Users can explore clustered TRIZ classifications, view the most frequently detected inventive principles, and analyze summary statistics. Interactive visual components facilitate deeper exploration and aid users in understanding trend patterns. Beyond assisting in document classification, this page offers a broader perspective on TRIZ principles identified across the system's test dataset.

Interface 3: Upload Page

The Upload Page, shown in Figure 3.1.6, enables users to submit patent documents in PDF format. Upon upload, the backend system automatically processes the file, performs text cleaning, classification, summarization, and stores the results in the database. Once the upload is successful, the user is automatically redirected to the Chatbot Page to interact with the processed content.

Interface 4: Chatbot Page

This page, shown in Figure 3.1.7, hosts the system's interactive chatbot, powered by a Retrieval-Augmented Generation (RAG) framework. Users can engage in contextual queries based on their uploaded documents and receive responses that are grounded and semantically relevant. Additionally, the page includes buttons that allow users to explicitly trigger summarization and TRIZ classification, offering an efficient way to review processed outputs.

3.2 Results Achieved

The final web application successfully integrates several key features, including TRIZ principles classification, automated summarization, and an interactive retrieval-augmented chatbot. These components were designed to support users in efficiently understanding and navigating the complex ecological patent document they have uploaded. Throughout development, the system was iteratively evaluated through internal testing, performance benchmarks, and informal user feedback to ensure it met both functional and usability goals.

One of the most significant components is the chatbot module, which enables users to query uploaded patents in natural language and receive meaningful, document-grounded responses. As this feature plays a central role in enhancing accessibility and user engagement, we conducted a focused evaluation of its performance.

Multi-Label Classification Evaluation

To assess the effectiveness of the multi-label classification module we designed, we perform a series of evaluations and metric aggregations, as outlined by Herrera et al. (2016). In here, we generate a multi-label confusion matrix and calculate Label-based Metrics. Ground truth labels were automatically generated using a hybrid method combining semantic similarity and zero-shot classification. Patent claims were embedded and compared to TRIZ principles, narrowing down to the top candidates. A language model then selected applicable principles among them, enabling scalable, consistent label generation without manual annotation for each patent claim. Figure 3.2.1 and Figure 3.2.2 show the result of the evaluation:

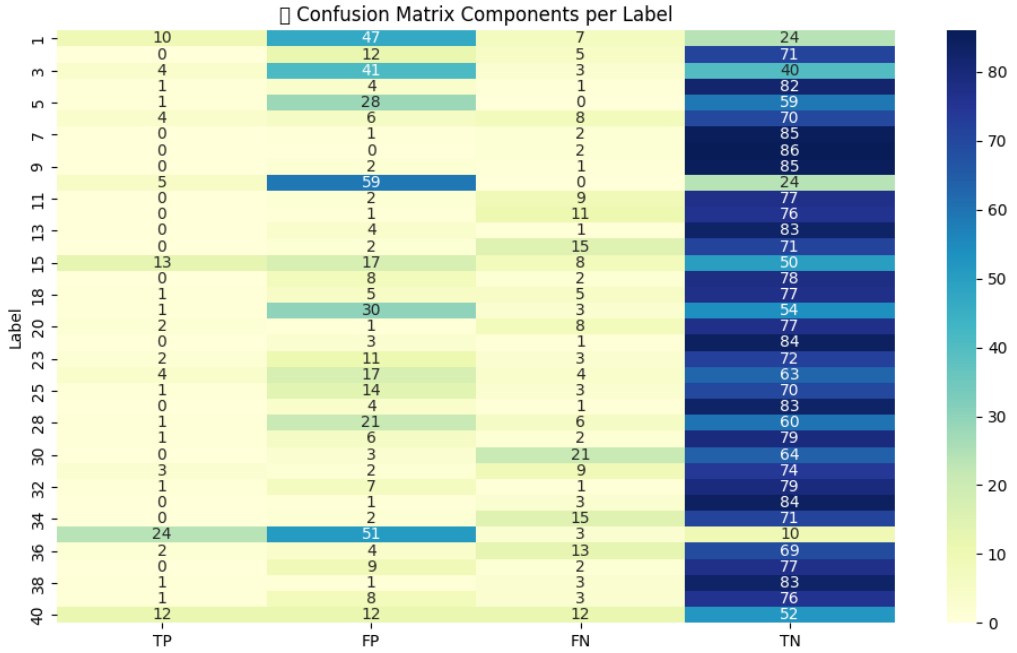


Figure 3.2.1: High-Level Heatmap of Confusion Matrix

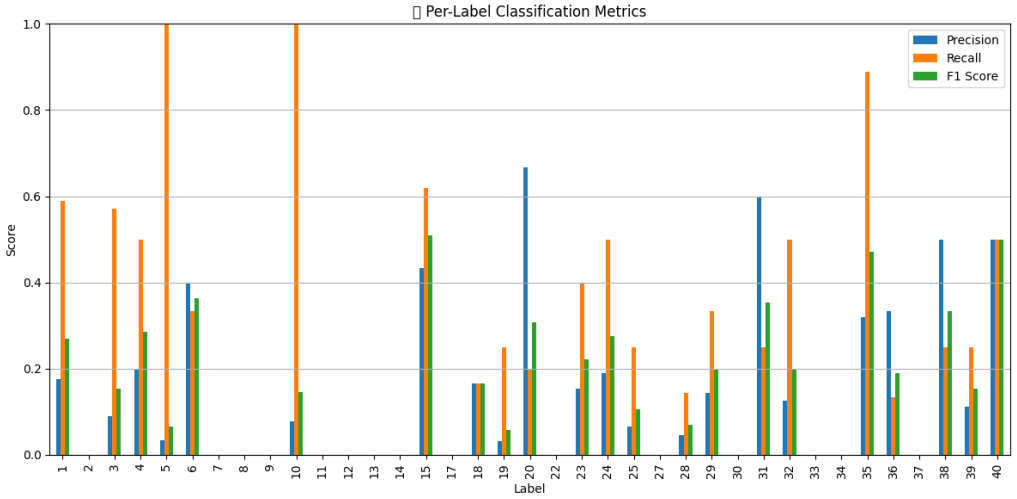


Figure 3.2.2: Label-Based Aggregate Metric

Unfortunately, both the high-level confusion matrix heatmap and the selected label-based metrics indicate that the model performed poorly compared to the ground truth labels we prepared. To explain the observed result, the sparse distribution in the confusion matrix suggests that the model either fails to capture the relevant TRIZ principles or produces very few correct predictions (true positives). Moreover, a low recall combined with high precision implies that the model is highly accurate when it does make predictions, but it misses a significant number of true positives. On the other hand, a high recall with low precision suggests that while the model successfully identifies most relevant labels, it also predicts many irrelevant ones, leading to over-labeling. Lastly, the F1-score, which balances precision and recall, serves as a more reliable overall measure of model performance, particularly important given the imbalanced nature of our dataset.

Chatbot Evaluation

To assess the chatbot's effectiveness according to its intended design, specifically, providing factually grounded responses based on the source of the patent document it loads, we conducted an evaluation using the RAGAS framework (Es et al., 2023), as outlined in the background. We achieve this by utilizing the WikiEval Dataset, a publicly available benchmark as stated by Es et al. (2023).

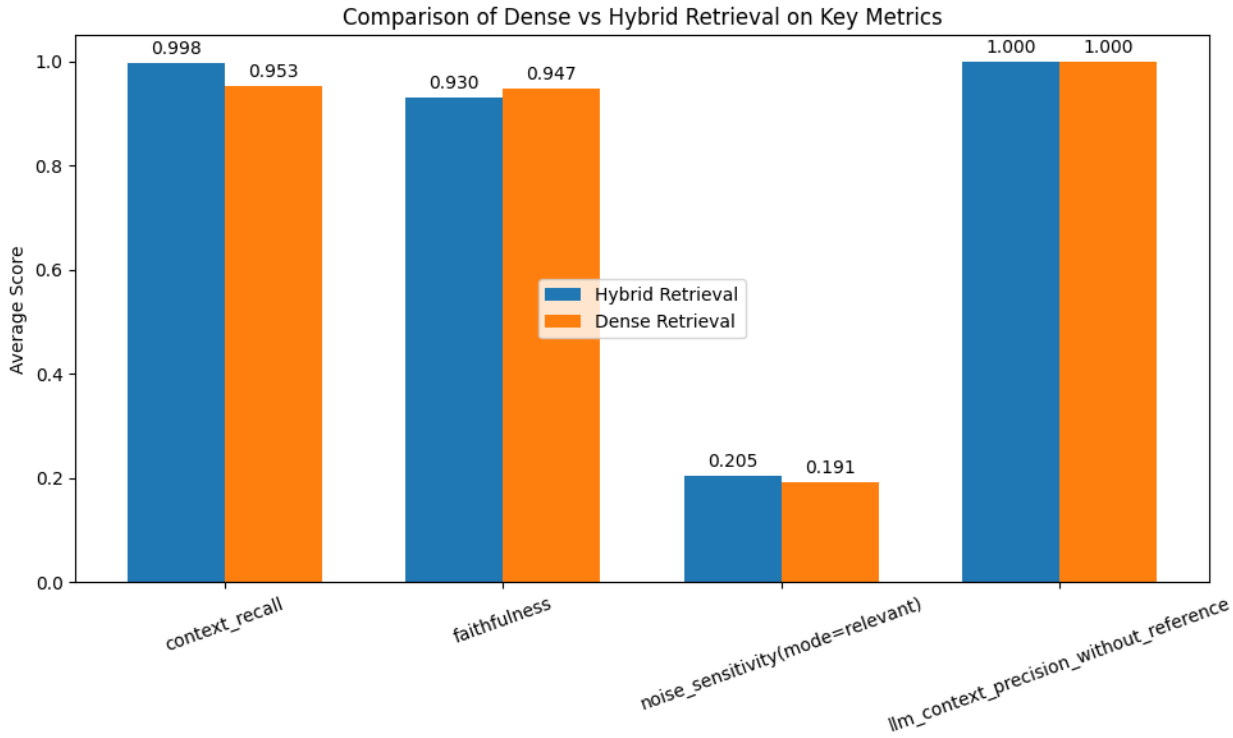


Figure 3.2.3: Comparison of RAGAS Metric Across Dense & Hybrid Retrieval

Our evaluation, as shown in Figure 3.2.3, revealed that the Hybrid Retrieval method demonstrated improved performance in retrieving relevant information, as evidenced by a higher context recall score. However, this improvement came with a trade-off in faithfulness, where dense retrieval alone outperformed the hybrid approach, producing responses that were more closely grounded in the original factual content. Despite this trade-off, we conclude that the system successfully achieves its primary goal: building a question-answering (QA) model grounded in factual documents. The final system is capable of responding to user queries accurately by leveraging relevant and trustworthy information from the provided sources.

3.3 How Are Requirements Met

Our project was guided by a defined set of functional and non-functional requirements, established during consultation with our supervisor, which aligned with the needs of end-users and the project's domain focus. These requirements were designed to ensure the final application delivered practical value while remaining technically feasible within the given timeline.

To monitor progress and ensure accountability, we maintained an updated Requirements Traceability Matrix (RTM) (Table 3.3.1) and a defined measurable User Acceptance Criteria (Table 3.3.2), both are detailed in 8.0 Appendices. These tools provide a clear mapping between

each original requirement and its implementation status, offering a structured view of which objectives have been fulfilled, are in progress, or remain pending. Table 3.3.3 below highlights several key requirements that have been successfully addressed in the final implementation:

Requirements	Functional or Non-Functional	Acceptance Criteria
Extract key information from patents to support summarization and TRIZ classification using the model	Functional	The system extracts keywords and sentences that represent core ideas of the patent, evaluated by manual inspection and being outputted inside chatbot.
Classify patent documents into one or more of the 40 TRIZ inventive principles	Functional	Classification achieves a minimum confidence threshold of 80% on labelled validation data, and accurately maps to applicable TRIZ principles.
Provide users with a web interface to upload patents and receive classification and summary outputs	Functional	Users can upload a patent and receive both a summary or TRIZ classification within 5 seconds after upload success upon clicking the buttons.
Enable interactive querying of processed patents through a chatbot interface	Functional	Users can ask context-aware questions about uploaded patents, and receive accurate responses within 5 seconds
Ensure the system can scale to handle large volumes of patent documents within the retrieval-augmented generation (RAG) framework without performance degradation.	Non-Functional	The system supports embedding and storage of up to 100 patent documents in the vector database, while maintaining retrieval latency under 3 seconds per query and consistent chatbot performance.
Provide responsive and accessible user interface for non-technical users	Non-Functional	Users with no technical background can navigate the platform, upload documents, and receive results without external guidance. Interface elements are clearly labelled, responsive, and instructions are stated.

Table 3.3.3: Overview of requirements successfully addressed

3.3.1 Mult-Label Classification System

This feature leverages the instruction-following capability of a decoder-based LLM to automatically classify patents according to TRIZ40 principles. It processes the text from the Text Processing Module through a structured, multi-step LLM pipeline designed to mirror how engineering experts identify inventive principles in ecological patterns.

Each step in the classification process is tailored to understand the technical language and context of the patent, allowing the model to assign multiple relevant TRIZ principles with contextual

explanations. This enables users to uncover the inventive strategies embedded in patents, supporting research and innovation analysis.

To address the non-deterministic nature of decoder-model outputs and enhance accuracy, we employ several strategies. One involves adjusting hyperparameters, such as temperature, top-k sampling, and max token length, at different pipeline stages to guide the model's behaviour. Another is the use of structured prompts and input formats to steer the model toward consistent and accurate results. Additionally, we enrich the pipeline with pre-prepared contextual knowledge on ecological domains. This domain-specific context helps the model make more reliable, accurate classifications aligned with the patent's subject matter.

By combining prompt design, parameter tuning, and contextual injection, the system improves both the repeatability and accuracy of TRIZ classification, making it a valuable tool for analyzing innovation embedded in technical documents.

3.3.2 Retrieval-Based Chatbot

This feature enables interactive exploration of patent documents using a Retrieval-Augmented Generation (RAG) framework. The chatbot is powered by a vector database that indexes both full-text patents and their structured components (e.g., claims, abstracts, classifications). By retrieving semantically relevant text chunks in response to a user query, the system provides grounded and context-rich responses directly supported by source material.

One of the core features that we pay heavy attention to in this is its ability to scale and generalize across large corpora of patents while maintaining explainability. By linking answers to specific document passages, the chatbot supports traceability and user trust. We can still maintain this groundedness in factual knowledge and achieve a good processing time through several optimization methods we have employed, fulfilling the requirements of a 3-second processing time.

3.3.3 Web Application

The web application serves as the primary interface for user interaction. It integrates all core functionalities, including file upload, classification, summarization, and chatbot interaction.

Throughout development, the frontend application was tested to ensure smooth navigation across all interfaces and consistent response times for key actions. These include transitions such as automatic redirection from the Upload Page to the Chatbot Page and real-time output rendering after document processing. The system consistently met performance expectations, with classification and summarization results returned within the defined 5-second threshold.

To ensure reliability, error-handling mechanisms were implemented to address common user-side mistakes, such as uploading an invalid file format or submitting an empty query. In each case, the

system responds with clear and informative messages, maintaining a stable and fault-tolerant environment. This contributes to a frictionless user flow that aligns with non-functional expectations for robustness.

The user interface was designed with accessibility in mind. All critical functions are accessible within one or two clicks, and instructions are presented. This ensures that even users with no technical background can navigate the platform effectively.

Additionally, the integration of all backend services into a unified front-end workflow satisfies the requirement, confirming that the system supports a complete end-to-end user interaction, from document upload to TRIZ classification and contextual querying without error or disruption.

3.4 Justification of Decisions Made

3.4.1 Decoder Model Selection

This decision is important as a decoder model primarily powers our app to give out answers to users. After thorough evaluation, we selected OpenAI's GPT-4o as the core language model for this project. Our application requires a nuanced understanding and classification of ecological patents based on the TRIZ40 principles, which demands advanced comprehension of technical language and contextual reasoning. GPT-4o's summarization and scientific knowledge, with their mature prompting capabilities, make it well-suited for this complex multi-label classification and summarization task.

We considered several open-source alternatives; however, most require significant computational resources that exceed our local infrastructure, necessitating third-party hosting, which would compromise privacy and transparency. Moreover, open-source models currently lack the refined language understanding and prompting guides that GPT-4o offers, which are essential for our classification accuracy.

Despite its advantages, GPT-4o's closed-source nature introduces challenges, including limited transparency on training data, input data privacy concerns, and cost implications. Nonetheless, given our technical requirements and project constraints, GPT-4o offers the most effective balance between performance, usability, and integration feasibility.

3.4.2 Separate Frontend-Backend Design

We adopted a separate frontend-backend design to improve modularity, scalability, and maintainability of the system. By decoupling the user interface from the backend logic and data processing, each component can be developed, tested, and updated independently without affecting the other. This separation also allows for easier integration of different frontend frameworks or backend services in the future, facilitating flexibility for future enhancements. Overall, it ensures a cleaner architecture and better adaptability as requirements change.

3.4.3 Chatbot Interface to Help User Inquiry

The main objective of our project's chatbot interface is to provide users with an intuitive, interactive means to explore and understand complex patent documents efficiently. To fulfil this, we decided to create an easy-to-use interface that can be used by domain experts in patent documents or even novice to understand and gain specific insights in the patents that they uploaded through a conversational experience. Notably, this chatbot interface was not part of the original project proposal but was introduced later.

3.5 Discussion of Project Results

Overall, the implementation of our project has made a meaningful contribution toward achieving our objective of assisting users in understanding the Eco-Patent document, as detailed in the Requirements Traceability Matrix (RTM) (Table 3.3.1) and User Acceptance Criteria (Table 3.3.2).

Automation and Explainability

The multi-label decoder-based classification enabled automated tagging of Ecological patent documents, specifically mapping claims to relevant TRIZ40 principles, reducing time spent to understand the patent from the lens of the TRIZ framework, equipped with an explanation of each principle and its correlation. Additionally, we implemented a retrieval-based chatbot system to support users in navigating and understanding patent content more intuitively. By grounding responses in the actual text of uploaded documents, the chatbot enables users to ask natural language questions and receive relevant, context-aware answers, thereby reducing the need for manual search through lengthy patent texts.

User Interface

Throughout this project, we made design choices, such as providing clear instructions, a user-friendly homepage, and meaningful exception handling, to ensure the application is accessible to both technical and non-technical users. These design considerations enable the abstraction of complex backend services, such as document processing, classification, and the chatbot, presenting only the necessary results to the user. This helps maintain a smooth workflow within the app, minimizing confusion and avoiding unexpected behaviours.

3.6 Limitations of the Project

3.6.1 Inexpertise in the TRIZ Framework

As a computer science student, I have limited expertise and practical experience with the TRIZ framework, a specialized methodology used for inventive problem-solving and patent analysis.

This lack of deep domain knowledge means that interpreting and applying the 40 TRIZ principles accurately can be challenging. We attempted to mediate this issue by applying methods and knowledge from reputable papers; however, collaboration with actual domain experts is expected to enhance the performance of this application further.

3.6.2 Choice of Document Processor: Selenium vs OCR

Our document processing module does not utilize OCR or deep content recognition, as these technologies require significant computation, lengthy inference times, or costly third-party services, all of which are beyond the scope and budget of this project. Instead, we rely on parsing HTML versions of patent documents available through the USPTO database. However, this approach depends heavily on the stability and structure of the USPTO's web pages. During development, frequent changes in database access methods and HTML structure forced us to repeatedly refactor the Selenium-based scraper to maintain accurate data extraction. This fragility impacts the reliability of the processing pipeline and remains a key limitation of the system.

3.6.3 Dependence on Closed-Source LLM Model

Our system relies heavily on GPT-4o, a powerful but closed-source language model. While it excels in technical understanding and mature prompting capabilities, its lack of transparency limits our ability to assess biases or errors. Additionally, data privacy concerns arise because the model processes input via external API calls, which means sensitive patent information is transmitted to third-party servers over which we have limited control. This dependency impacts scalability, introducing latency per call and preventing concurrent processing.

3.7 Improvements and Future Works

- **Model Choices**
  In future iterations, we aim to experiment with a self-hosted open-source LLM model that already offers the same technical capabilities as LLM, or a smaller parameter model that we fine-tuned on scientific knowledge and previous answers from the current iterations. We also wish to enable multi-model options to be used by different users.

- **Better Multi-Label Classification Module**
  Our current module relies heavily on a few-shot prompts, executed through sequential API calls to a decoder-based model. A more scalable and efficient method will be through a fine-tuned decoder model trained on the pipeline tracing data we generated from the current iterations of our module. This data includes input patterns, corresponding final output, and intermediate results. With a sufficient volume of high-quality data, we could train a model

that internalizes the patterns and logic required for tasks such as multi-label classification and structured reasoning on ecological patents.

Benefits of this approach:

1. **Improved Inference Speed:** Eliminates the need for sequential few-shot prompting, resulting in faster response times.
2. **Domain-Specific Accuracy:** Enables fine-tuning to ecological patent language and TRIZ principles, producing more relevant and specialized outputs.
3. **Multi-User Scalability:** A smaller, fine-tuned model can be deployed across clusters, supporting high request volumes from multiple users or batch processing scenarios.

- **OCR-Based Doc Processing**

  Our current iteration of the Document Processing module operates by utilizing HTML version data retrieved from the USPTO database, which we parse into distinct sections. While functional, this approach is not ideal for enterprise-level applications due to its fragility against changes in HTML structure and reliance on web scraping, which we highlighted in this report.

  In future iterations, we can implement third-party services such as Google Cloud Vision or Mistral's Document.AI for more robust document parsing. The benefits of this are:

  1. By integrating advanced document processing solutions, we are no longer restricted to the USPTO's specific data structure. This enables the system to handle patents from different patent offices around the world, each of which may use varied formats and structures.
  2. Enterprise-level scalability and reliability could enable smoother integration in production environments.
  3. The ability to support patent design and schema, where advanced document processing solutions allow the system to recognize and adapt to different patent layouts, including utility patents, designs, and schema-specific documents. This enables source-referral in the chatbot retrieval to specific pages and schema.

- **Future Chatbot Innovations**

  In future versions, we plan to build a unified chatbot interface that handles all user interactions, like classification, document uploads, 40-TRIZ queries, and search, through a single input box. This will streamline the user experience by removing the need for separate buttons or workflows. To enable this, we will implement an Encoder Router model that identifies the type of user input and routes it to the correct backend module for processing. We also aim to enhance the chatbot's retrieval system by incorporating a verified information index from trusted sources, including textbooks, research papers, and patents. Users will be able to trace answers back to exact sources and extra pointer.

- **Website Hosting**
  Finally, to serve users, we need to deploy this application into an actual web-accessible environment. This involves hosting the frontend interface, backend services, and database components on a reliable platform.

  To do this, we are required to implement:
  1. **Safety Measures in the Frontend:** Implement security features such as input sanitization, rate limiting, and authentication tokens to prevent malicious actors from exploiting the user interface or triggering unintended API calls.
  2. **Production-Ready REST API:** Refactor the current backend into a scalable REST API architecture that supports concurrent requests. Load balancing mechanisms must be integrated to evenly distribute traffic and ensure stable performance under user demand.
  3. **Backend and Frontend Deployment:** Host the application using cloud services. This includes setting up automatic deployment pipelines and ensuring proper integration between the frontend and the backend.
  4. **Multi-Cluster Setup:** Deploy the services across multiple server clusters or containers (e.g., using Kubernetes or Docker Swarm) to support parallel handling of user queries, especially those interacting with LLM models or computationally intensive tasks.
  5. **Proper DevOps Infrastructure:** Establish a DevOps pipeline, CI/CD tools, and robust testing frameworks. This will ensure reliability, maintainability, and fast recovery in case of failures.

3.8. Critical Discussions on Outcomes

On the technical side, we proposed a multi-label classification approach using a few-shot prompts and multi-hop reasoning. Initially, this concept was not grounded in the TRIZ framework or its 40 inventive principles. However, during development, we connected the system to a relevant research paper involving ecological patents and TRIZ, allowing us to contextualize the model's predictions more effectively. As we progressed, we faced issues with stability and accuracy, which led us to propose a refinement: incorporating reasoning trace data along with hyperparameter tuning to improve consistency. For evaluation, we moved beyond a basic confusion matrix with a label-based aggregation metric; however, our model performs poorly compared to the ground truth. We believe this is due to the lack of expert-annotated TRIZ40 labels, which led us to use automated labeling, prone to overfitting and misalignment with the task post-integration. This is the most immediate improvement technically.

On the application side, our original design included a dedicated patent database engine to optimize efficiency and reduce redundant processing. It is also planned for semantic similarity search using vector embeddings and keyword search via Elasticsearch to support exploration and retrieval. However, we later decided to take a different approach, focusing on a chatbot interface as the

primary user interaction point. This shift prioritizes conversational access to patent insights, aiming for a more intuitive and accessible user experience. We decided that this feature better fulfills our main objective of helping both domain experts and newcomers understand patents more easily and effectively.

Additionally, even though our project fulfills the requirements and objectives outlined, there is still room for improvement in terms of the application's readiness for production. Key areas include performance optimization, scalability to support multiple concurrent users, and robustness in handling diverse real-world scenarios. At its current stage, the application functions well as a proof of concept and meets the scope of this project, but further development is needed for it to be considered production-ready. Recognizing these limitations gives us a clear direction for future work.

4.0 Methodology

4.1. Design

The final design of the application is illustrated in the following application flow diagram (Figure 4.1.1).

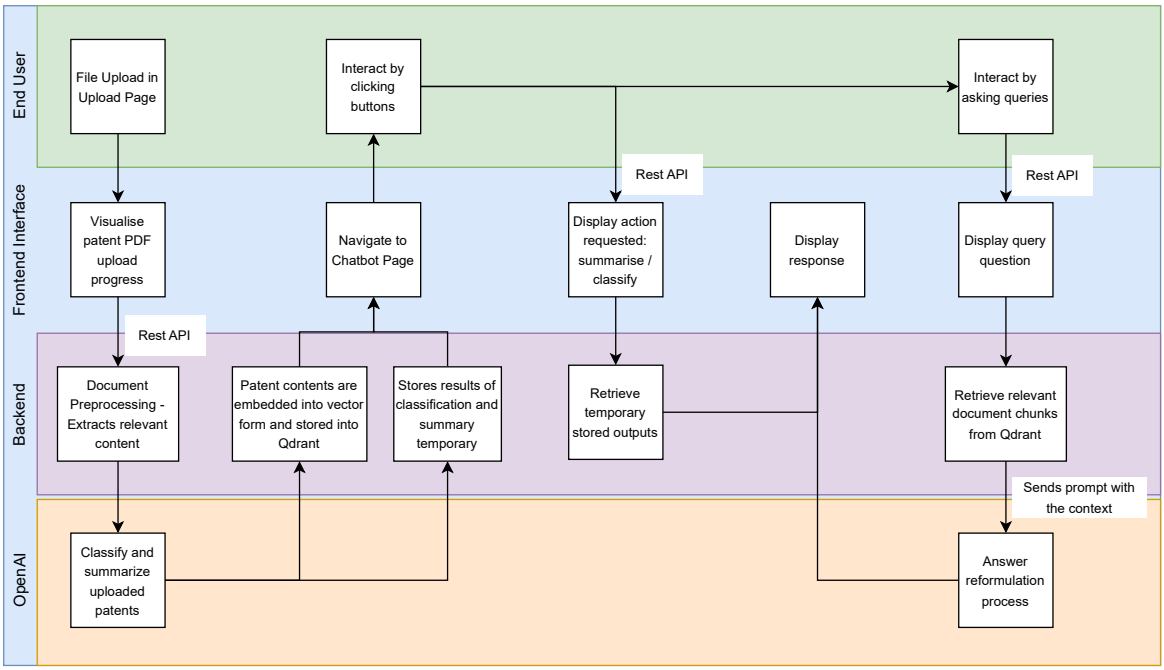


Figure 4.1.1: Application Flow Diagram

Our application is structured into two primary high-level modules: the frontend and the backend. These modules communicate via REST APIs to enable seamless data exchange. Additionally, the system relies on several external dependencies, including third-party language models (OpenAI), a vector database (Qdrant), and a headless browser for automated HTML retrieval.

4.1.1 High-Level Explanation of Application Flow

Users begin by cloning the GitHub repository and deploying the application via a NodeJS command, which starts both frontend and backend services in containerized environments for stability. By opening the assigned localhost URL in a browser, users land on the Home Page, which introduces the app's purpose, key features, and usage instructions to help them get started. From there, users can explore the Infographics section, where six months of analysis and key insights are visually presented to summarize trends and important findings.

Next, on the Upload Page, users are guided to follow instructions carefully to upload patent files that meet the project's scope. Once a file is selected, the frontend sends it via a REST API POST request to the backend. The backend processes the file through the Document Processing module, which converts it into structured text, then Classification module classifies it according to predefined criteria. A basic summarization is also generated to help users understand the document's content.

After processing, the backend returns results in JSON format, triggering the frontend to automatically redirect users to the Chatbot Page. Here, users can interact with the chatbot by typing queries, which are processed semantically against the stored classification data, enabling smooth, ongoing conversations about patent insights.

4.1.2 Deviation from Proposed Design

The original application flow diagram from our previous project proposal is included in the Appendix. In our proposed design, we intend to implement a dedicated database engine to store patents uploaded by different users. This approach was envisioned to optimize system efficiency and reduce redundant processing. For example, when a user uploads a patent that already exists in the database, the system can immediately retrieve the pre-processed data, including classification results, summaries, and embeddings, without needing to reprocess the file through the full backend pipeline. This significantly reduces response time and resource consumption. With this database system in place, we also aimed to enable two additional supporting functionalities:

1. **Similarity Search**

Leveraging the stored vector embeddings of processed patents, users would be able to perform semantic similarity searches. This allows the system to recommend patents that are closely related to embedded content. The benefit is that users can explore variations or prior art without having to manually search through raw patent data.

2. **Keyword Search with Elasticsearch**
   In addition to semantic similarity, we proposed integrating Elasticsearch to allow users to perform fast and scalable keyword-based searches across the database. This supports use cases where users are looking for patents containing specific technical terms, product names, or invention descriptions.

4.2 Tech Stack

The following tables show the minimum hardware and software requirements necessary to develop, test, and deploy our application. This includes not only the baseline system specifications but also outlines key library dependencies and external services required for full functionality.

4.2.1 Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
CPU	2 cores	> 4 cores
RAM	4 GB	> 8GB
Storage	5 GB	> 10 GB
Internet	Low-latency	> Medium-latency
GPU	Not required	Min GPU
OS	Ubuntu Linux 16.04 LTS, MacOS 13.7.4, Windows 8	Ubuntu 24.04.2 LTS, MacOS 13.7.4, Windows 11

Table 4.2.1.1: Hardware Requirements

4.2.2 Software Requirements

Here are the software applications used in developing both the frontend and backend into one coherent application.

Component	Version	Function
Docker	28.1.1	To containerize backend and dependencies align the correct way
Docker-compose	2.32.4	To connect and run different containers we need

NPM	10.9.2	Manage dependencies in Node.js
Node	22.15.0	Runtime to configure and load frontend interface codebase
Git	2.43.0	Versioning system we use
Python	3.13	Backend and Model Programming Language
TypeScript	5.5.3	Frontend Language

Table 4.2.1.2: Software Requirements

To fulfill our objective of processing ecological patents through the lens of the TRIZ framework, we're developing a tailored application that integrates and extends the capabilities of decoder-based large language models, leveraging and customizing a core set of specialized libraries in Python.

Library	Version	Usage	Location in System
BeautifulSoup4	0.0.2	HTML parsing for data extraction	Data Extraction
Cohere	5.15	LLM access for embeddings & reranking	LLM Integration / Semantic Search
Dotenv	0.9.9	Loads secret environment variables	Configuration
Flask	3.1.0	Backend REST API framework	Backend API
Langchain	0.3.25	LLM orchestration for chatbot flows	Chatbot / LLM Orchestration
OpenAI	1.76.2	Accesses OpenAI LLMs (gen, summarization)	LLM Integration
Pydantic	2.11.4	Data validation & API schema handling	Data Handling / API
Qdrant-client	1.14.2	Vector database client for RAG/search	Vector Database / RAG
Selenium	4.31.0	Web browser automation for scraping	Web Scraping
Together	1.5.6	Accesses Together AI LLMs	LLM Integration
TQDM	4.67.1	Progress bars for iterative tasks	Utility

Table 4.2.1.3: Libraries

To achieve the sophisticated functionalities of our patent analyzer and TRIZ-based ecological patent processing, our system relies on a robust set of external dependencies that must be run alongside the underlying libraries.

Library	Usage	Services Used
Cohere	Reranking and Embeddings model	rerank-english-v3.0, embed-english-v3.0
OpenAI	Decoder model and Encoder model	GPT-4o, Ada-002
Qdrant	RAG retrieval and Patent Classification context injection	Qdrant Vector Database Cluster
Together	Accesses Together AI LLMs	DeepSeek R1 end points

Table 4.2.1.4: External Dependencies

4.3 Design Implementation

4.3.1 Document Preprocessing

As mentioned in Section 4.1.1, our application begins its workflow by processing patent files submitted by our users through the frontend's upload page, into a string format that we can divide and process further to services we implemented beforehand for specific tasks.
Due to the inconsistent structure of USPTO patent PDFs and the absence of an essential section, such as claims, we decided to utilize the HTML version of each document, accessed using a Selenium-based scraper. This method provides more structured and extractable content. Here are the steps we performed:

- Extract Patent Code: Open the PDF file and retrieve the file metadata. Use the Python **regex** library to parse through the metadata and extract the appropriate patent code.
- Automated Access to USPTO database: Initialize a headless **Selenium** browser and input the correct link and flow to open the USPTO public search directory. Input the extracted patent code into the search bar and click the result link. Load the HTML file found in the link.
- Download the HTML file and initialize a **BeautifulSoup4** instance to parse the document into different sections.
- Load a **Pydantic** Data Structure, standardize the output for further development, and use different modules.

The most challenging aspect of the implementation was automating the retrieval of HTML files from the USPTO's basic search directory using Selenium. This complexity arises because the HTML content is protected by a session-based JWT token that changes dynamically with each visit. As a result, direct access to the file was not feasible.

4.3.2 Multi-Label Classification

As mentioned in Section 3.0, our system has a multi-label classification component designed to help expert users understand eco-patents through the TRIZ lens, particularly the 40 TRIZ Inventive Principles. This task is classically framed as a multi-label classification problem, where a single patent may correspond to multiple inventive principles simultaneously. Conventionally, the solution to such problems involves using an embedding-based classifier, as these models excel at capturing complex semantic and textual relationships. However, this approach is not feasible in our case due to the niche nature of our ecological patent dataset, which lacks sufficient volume and diversity to train such models. As a result, we decided to adopt a classification using few-shot sequential prompting through a decoder model, inspired by a manual 40-TRIZ principles classification methodology. We aim to explore whether a decoder model can be effectively adapted and prompted to produce deterministic classification results, despite this being contrary to its inherent generative nature.

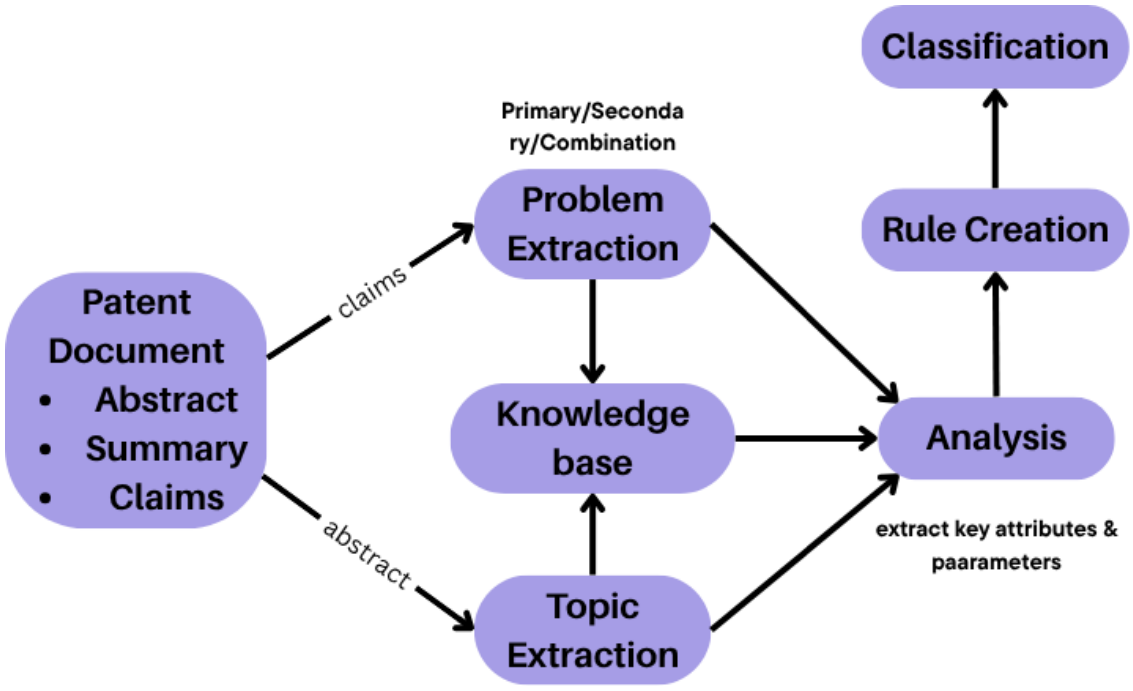


Figure 4.3.2.1: Classification Module's Flow Diagram

Steps taken in the classification pipeline (Figure 4.3.2.1):

- Start by passing the PatentDocument (a Pydantic model) from DocProcessing.
- Extract primary and secondary problems from the Claim using a structured prompt.
- Apply a guardrail prompt to determine if the problem is ecologically relevant.
- If valid, generate a long-form analysis using analysis prompt.
- Use this analysis to derive actionable guidelines with the rule creation prompt.
- Combine the Claim and generated rules into the classification prompt.

- Obtain the final TRIZ classification and a descriptive explanation.
- Standardize the output using ClassificationPipelineOutput for modular integration.

During development, we observed two major issues with traditional multi-hop reasoning using sequential few-shot prompting: inconsistent outputs and loss of context across steps. These led to unpredictable results, such as fluctuating TRIZ principles classifications when processing the same patent multiple times.

To solve this, we introduced a **reasoning trace injection** method. This involves adding structured reasoning steps directly into the classification pipeline to help the model stay consistent, use earlier inferences, and improve accuracy.

Our approach builds on recent advancements in reasoning-guided prompting:

- Wei et al. (2022) demonstrated that intermediate *Chain-of-Thought* (CoT) steps significantly improve reasoning accuracy in complex tasks.
- Johnson (2025b) showed that *reasoning step distillation* enhances the performance of smaller decoder models on logical reasoning benchmarks.

Rather than relying on full model fine-tuning or LoRA adaptations, we adopt a parameter-efficient strategy by injecting curated reasoning traces as contextual prompts, preserving consistency across reasoning stages.

To improve the accuracy and interpretability of multi-hop reasoning in patent classification, we propose a novel approach that synthetically generates and injects correlated reasoning trace data into the classification pipeline. Our methodology involves the following steps:

1. **Synthetic Reasoning Trace Generation**
2. We utilize the DeepSeek R1 model (hosted via TogetherAI) to generate reasoning traces using the QA dataset retrieved from Huggingface, simulating multi-hop reasoning.
3. **Vector Embedding and Storage in Qdrant:** Each reasoning trace is cleaned and chunked using Langchain tools, then embedded into a high-dimensional vector space via OpenAI's Ada-002 model with original trace and domain category included inside.
4. **Reasoning:** To align with the sequential and compositional nature of multi-hop reasoning, the retrieval process is adapted to perform targeted searches.

Steps Overview:

1. **Dynamic Weight Generation**
   A dynamic weight is assigned to each extracted problem based on its tier, reflecting its importance or complexity. This weight guides retrieval and reranking in later steps.
2. **Problem Embedding and Targeted Retrieval**

Problems are embedded using OpenAI's Ada-002 model and queried against Qdrant. Payload filters ensure domain-relevant results based on the extracted topic.

3. **Search Result Weight Adjustment**
   Retrieved results are adjusted by applying dynamic weights to similarity scores, prioritizing traces linked to higher-tier problems.
4. **Reranking for Relevance and Coherence**
   Cohere's rerank-english-v3.0 model reorders the results, combining semantic relevance with weight scores in a 70:30 ratio. (See Figure 4.3.2 for formula)
5. **Classification Integration**
   Top-ranked reasoning traces are injected into the classification pipeline as contextual input, enabling enriched multi-hop reasoning for improved accuracy and interpretability.

4.3.3 Retrieval-Based Chatbot

To complement the Multi-label Classification module, we developed a chatbot that allows users to query their uploaded patents and receive document-based answers. It uses a retrieval-based approach powered by Qdrant, a vector database supporting dense and sparse search, payload filtering, and hybrid retrieval. Our hybrid strategy combines BM42 for keyword matching and Ada-002 for semantic understanding, optimized with Qdrant's prefetch settings to deliver fast, accurate, and context-aware responses.

Models Used:

- **BM42** is a sparse embeddings model that improves BM25 by addressing its limitations with short or non-repetitive texts. Instead of relying on raw term frequency, BM42 uses transformer attention, specifically [CLS] token attention weights, to assess each token's contextual importance, generating more semantically meaningful sparse embeddings (Vasnetsov, 2024).
- **Ada-002** is a dense embeddings model by OpenAI that captures semantic meaning in high-dimensional vectors. In our hybrid retrieval system, it encodes patent documents and queries for semantic search, enabling matches even when wording differs.

Unfortunately, hybrid search using high-dimensional embeddings like Ada-002 is computationally intensive. To optimize retrieval speed and scalability, we implemented two key techniques:

1. **Binary quantization** compresses dense embeddings into binary form, enabling 85× storage and 40× speed improvements through efficient bitwise operations like Hamming distance. This is especially effective for over-parameterized embeddings, retaining core semantic meaning while reducing overhead. (Blog Post)
2. **Reciprocal Rank Fusion (RFF)** combines results from multiple retrieval models by scoring documents based on their ranks (1 / (rank + k)), then summing scores across all

lists. This approach boosts documents consistently ranked highly, improving retrieval performance through model synergy. (Blog Post)

Steps taken to enable this:

1. Cleaned and chunked the P ydantic's instance content using Langchain.
2. Generated two types of embeddings (Ada-002 and BM42) for each chunk.
3. Configured a Qdrant collection to support hybrid vectors with Binary Quantization.
4. Inserted all embedded chunks into the Qdrant collection.

4.3.4 Backend Design

Our system is built on a RESTful API architecture to ensure smooth, stateless communication between the frontend and backend. This standardized approach allows backend functionalities to be exposed over HTTP, making the system scalable, maintainable, and compatible with future client applications we may introduce. To preserve data consistency and avoid concurrency issues, the API is designed to handle one file upload at a time. This ensures that validation, parsing, and storage occur in a controlled sequence, minimizing errors during processing. In addition, we implemented comprehensive exception handling to manage common runtime issues, such as invalid file formats, empty uploads, or unexpected processing errors. By returning clear and meaningful HTTP responses, the API not only improves system stability but also contributes to a smoother and more transparent user experience.

4.3.5 Frontend Design

Our frontend web application is built with React.js, enabling a dynamic, modular interface suited for real-time interactions. For styling, we use Tailwind CSS, a utility-first framework that ensures consistent, responsive layouts across devices while streamlining development. This combination allowed us to efficiently convert design prototypes into a user-friendly interface aligned with our intended experience.

The interface is structured into four primary views: Home, Insights, Upload, and Chatbot. Each serves a distinct user purpose, from welcoming users to presenting analytics, to submitting documents, and finally interacting with the system via the chatbot. Navigation is handled through a persistent left sidebar for seamless transitions between views. To enhance usability, we added interactive elements, including loading indicators during PDF uploads, error messages for invalid patent formats, and confirmation prompts to alert users when they attempt to leave the page.

The application integrates closely with the backend via a set of RESTful API endpoints, which handle file processing, classification retrieval, summarization, and chatbot queries. These endpoints return structured JSON responses, which are then displayed on the frontend.

Overall, the frontend was designed to be lightweight, responsive, and easy to navigate, with a strong emphasis on clarity and user experience.

5.0 Deliverables

5.1 Project Deliverables

Our project has developed a series of key deliverables to support efficient planning and execution throughout the semester. Among them are the User Guides and Test Report, which outline how the system operates and document the outcomes from unit, integration, and user acceptance testing. Meeting minutes were consistently recorded to capture the essential decisions made during team discussions and supervisor check-ins. To keep our progress on track, we used a Gantt Chart (Figure 5.1.1) for task scheduling and milestone planning. A Risk Register (Figure 5.1.2) was created to help us identify and prioritize potential issues early in the development process. We also structured our tasks using a Work Breakdown Structure (WBS) (Figure 5.1.3) to manage the project scope clearly and systematically. To ensure that all user and system requirements were met, we implemented a Requirement Traceability Matrix (RTM) alongside a set of clearly defined User Acceptance Criteria. (All deliverables can be found in the Appendices.)

5.2 Software Deliverables

5.2.1 Summary of Software Deliverables

The EcoPatent Analyzer reflects our team's vision of making advanced patent analysis accessible through intelligent automation. This end-to-end web application converts complex USPTO patent documents into actionable insights based on TRIZ principles. The core functionalities are implemented in the backend using Python, while the frontend design is customized with HTML and Tailwind CSS. Users can launch the interface locally by executing the `npm run dev` command inside the terminal.

5.2.2 Main Software Features

5.2.2.1 Patent PDF Upload

The platform supports uploading USPTO patent documents in PDF format via drag-and-drop or the file "Browse". This action triggers the backend processing pipeline, including document parsing, summarization, and classification.

5.2.2.2 Real-time File Validation

The application performs automatic validation checks as soon as a file is uploaded, and only valid files proceed to the processing phase:

- File format verification (PDF only)
- File size validation based on configurable limits
- PDF header parsing to verify document structure
- Patent number extraction using multiple regex patterns

5.2.2.3 Interactive Insights and Visualization

Implemented via the "ChartsSection" component, the application offers an interactive dashboard that displays visual summaries of classification results from test data. Users can explore TRIZ principles and observe patterns across patents, with hover-enabled tooltips for a deeper insight.

5.2.2.4 Interactive User Interface

- **Chatbot Interaction:** Users can query the uploaded patent through a conversational interface powered by RAG (Retrieval-Augmented Generation). The chatbot displays responses grounded in the document content and allows contextual follow-up questions.

- **Application Usage Guide:** A dedicated section on the Home Page guides users through the platform's features step-by-step. This "How It Works" section walkthrough ensures clarity for first-time visitors.

- **Error Handling Messages:** If the upload or processing fails (e.g., an unsupported file type or a missing patent number), the system returns user-friendly messages with clear instructions for resolution.

- **Confirmation Pop-up Message:** Before users exit the Chatbot Page, pop-up dialogues prompt confirmation to prevent accidental data loss or interruption of interaction.

- **Application Tech Stack:** A section on the Home Page visually lists and credits all libraries, frameworks, and APIs used in the development of the project. This provides transparency and highlights the key technologies that power the application.

5.2.2.5 Application Usage

Steps (a) to (d) illustrate the full software usage flow, guiding users through how to upload and interact with the application effectively. A detailed user guide is available here (User Guidelines) for further reference.

(a) Home Page - First Impressions and Onboarding

The Home Page immediately establishes trust and clarity, as mentioned in Section 3.1, it features our custom Hero component, a comprehensive How It Works guide, an FAQ, and a Tech Stack showcase. We designed this entry point to eliminate confusion for first-time users while providing clear pathways to analysis. Figures 3.1.1 to 3.1.4 demonstrate the Home Page.

(b) Insights Page

Navigating from the sidebar to the "Insights" page, users are presented with the Charts Dashboard, which transforms analytical results into clear visual insights. Implemented through our "ChartsSection" component, this page offers interactive visualizations that highlight TRIZ principles, distributions and innovation patterns. These insights enable users, particularly R&D strategists and patent analysts, to identify key trends across multiple patent portfolios in an intuitive and informative manner. Figure 3.1.5, Figure 5.2.2.5.1, and Figure 5.2.2.5.2 demonstrate the Insights Page.

(c) Upload Page - Streamlined Document Processing

Moving on to the "Upload" page, users can upload USPTO patent PDFs either by dragging and dropping them into the designated upload zone or by browsing their file system from Figure 3.1.6. During this process, the interface provides immediate visual cues, such as highlighted drop areas, progress indicators, and real-time status messages, including "Uploading…", "Processing…", and "Extracting patent number…". Upon successful extraction, confirmation messages display key patent details.

In the case of issues, users are presented with clear, actionable error messages, for instance, "Only PDF files are allowed" for unsupported formats or "Patent number not found" with specific pattern attempts listed for transparency, or "Upload failed. Please try again.", as shown in Figure 5.2.2.5.3.

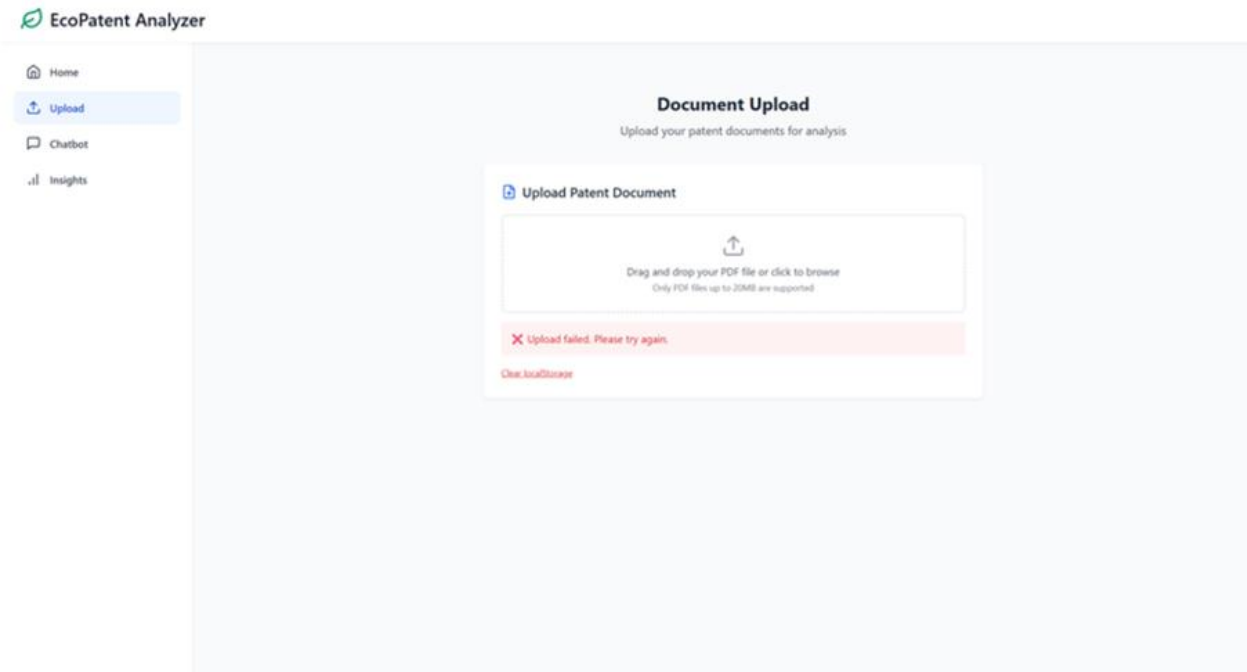


Figure 5.2.2.5.3: Patent Upload Failed

If the upload exceeds size limits, appropriate guidance is provided. Once a file passes validation and processing, the system automatically redirects users to the Chatbot interface (see Figure 5.2.2.5.4).

(d) Chatbot & Analysis Interface - Interactive Intelligence

After a successful upload, users can engage with the system by clicking either the "Summarise" or "Classify" buttons to receive contextual responses based on their uploaded patent document (refer to Figure 5.2.2.5.5 and Figure 5.2.2.5.6).

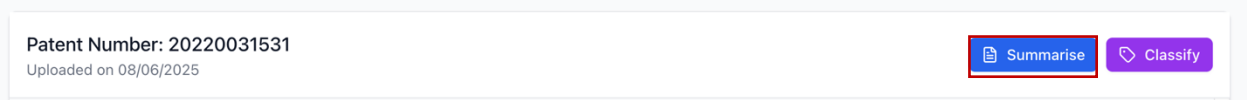


Figure 5.2.2.5.5: "Summarise" Button

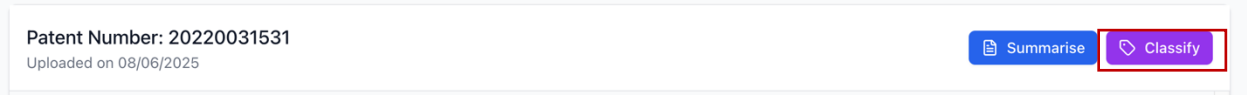


Figure 5.2.2.5.6: "Classify" Button

The "ChatInterface" component seamlessly integrates TRIZ classification results with natural language interaction, enabling users to explore patents through questions like "Why was Principle 15 detected?" or any patent-related questions.

5.2.2.6 Backend Architecture and AI Engine

Our Flask-based backend features two well-structured RESTful endpoints. The `/upload` endpoint manages the end-to-end analysis pipeline starting from patent parsing through the "DocProcessing" class, followed by TRIZ classification via the "PatentClassifier" service. The `/query` endpoint supports conversational interactions through the "PatentChatbot"; further details are in Section 4.3.3.

The classification engine stands as a key technical highlight in our implementation of explainable AI. A detailed explanation can be found in Section 4.3.2.

5.2.2.7 Supporting Infrastructure

We containerized the entire application using Docker and Docker Compose to ensure consistent deployment across development, staging, and production environments. The "compose.yaml" file includes service dependencies and health checks to maintain system reliability. Our project deliverables include comprehensive technical documentation (DOCUMENTATION.md), environment configuration templates, and dependency management via the "requirements.txt" file. Modular prompt templates are centralized in "prompt_template.py" to support maintainability and reuse. The repository is structured following professional software engineering standards, with a clear separation between frontend components, backend services, and infrastructure configuration. Figure 5.2.2.7.1 shows the overall architecture flow chart.

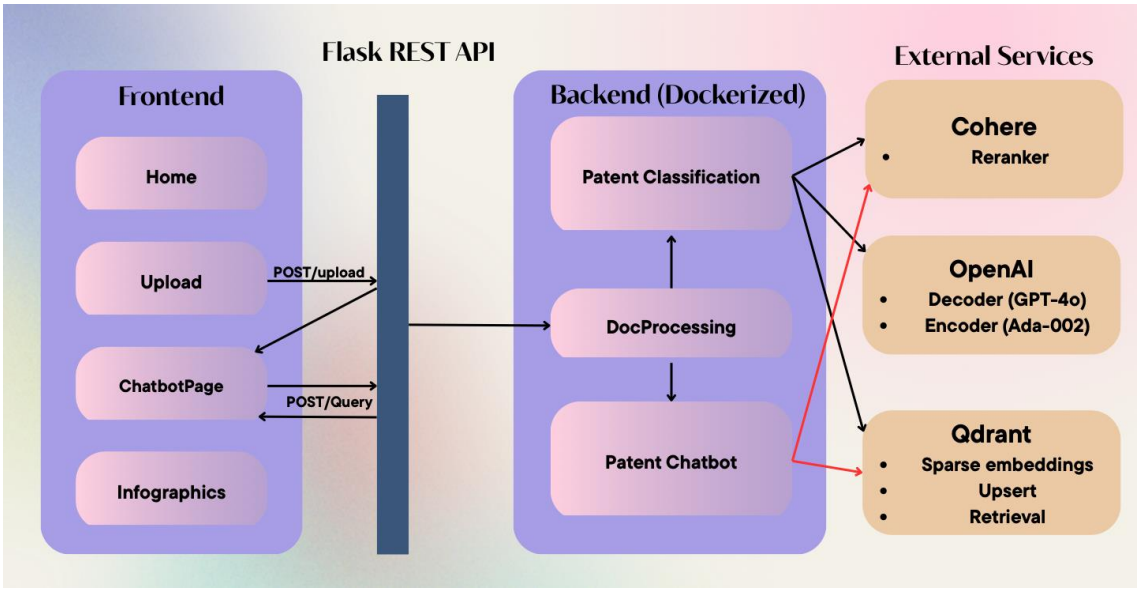


Figure 5.2.2.7.1: Containerized system architecture: modular backend, vector database, and frontend

5.2.3 Software Quality Summary

5.2.3.1 Robustness

We implemented comprehensive validation at every system boundary. File uploads undergo multiple checks: format verification, size limits, PDF header validation, and patent number extraction using resilient regex patterns. When extraction fails, users receive detailed debugging information: "Patent number not found - Tried patterns: US-<digits>-A1 and standalone 5–11-digit numbers."

Our classification pipeline employs defensive programming with try-catch blocks at each processing stage. Validation Error and general exceptions are handled gracefully, ensuring users receive meaningful feedback rather than system crashes. Global state management maintains robust document context throughout the user session.

5.2.3.2 Security

Security was prioritized from architecture to implementation. All sensitive credentials (OpenAI API keys, Qdrant database keys) are managed through environment variables, never exposed in our codebase. The chatbot interface includes guardrail mechanisms that prevent prompt injection attacks, while our Docker configuration utilizes minimal-privilege containers for enhanced isolation.

Our guardrail function validates ecological relevance before processing, preventing inappropriate content analysis through simple prompting. File processing occurs in-memory, when possible, with automatic cleanup protecting user confidentiality.

Although basic security features were implemented, several security shortcomings were identified. Ideally, our frontend and backend should incorporate a proper user verification system, which could include a signup/login mechanism or the use of JSON Web Tokens (JWT) to manage user sessions securely. This would ensure that user actions are authenticated and that access to features or data is appropriately restricted based on user roles or identities.

5.2.3.3 Usability

We designed for inclusivity and clarity. The persistent sidebar navigation with clear icons enables seamless workflow transitions. Our error messaging prioritizes plain language over technical jargon: "Only PDF files are allowed" rather than exception traces. Visual feedback through progress indicators and status messages keeps users informed throughout processing.

The chat interface provides action buttons for common tasks (summarize, classify), reducing cognitive load for new users. Navigation protection dialogs prevent accidental data loss during active analysis sessions. Lastly, we have added instructions and workflow guides throughout the app, including the Home page and the upload page.

An improvement we plan to implement soon is to combine all user interactions into a single interface through an encoder router that allocates input to different modules we prepared. We also plan to enhance the chatbot's responses by integrating it with trusted sources, such as textbooks, research papers, and patents. Users will be able to see where the information comes from, including page numbers and visuals.

5.2.3.4 Scalability

During development, we've made conscious design decisions to ensure our system can support a large number of concurrent users seamlessly. A significant part of this effort involves optimizing our RAG (Retrieval-Augmented Generation) system and developing strategic RESTful APIs to connect various modules. A high-level design choice was to adopt a container-based application to normalize this in the development flow, encouraging encapsulation and independent scaling in the future. We also followed Object-Oriented Programming (OOP) design principles for frequently used functions to ensure the code is modular, reusable, and easier to maintain. An example of this is our custom LLM caller class, which we implemented ourselves instead of relying on third-party libraries to control API call frequency.

Additionally, we anticipate ongoing challenges in achieving scalability due to the resource-intensive nature of our application. To address this, future versions will likely require a more advanced software architecture involving multi-container clusters, load balancing, and other distributed system techniques.

5.2.3.5 Maintainability and Documentation

We've built our codebase for long-term sustainability by separating functions into modular services, isolating components such as patent parsing (`patent_parser.py`), text classification (`textClassification.py`), chatbot features (`chatbot.py`), and LLM interfaces (`LLM.py`). We utilize type hints and Pydantic to enhance code readability and debugging, while centralized prompt management (in `prompt_template.py`) facilitates easy updates. Our REST API design ensures stateless, resource-based communication for scalability and interoperability. This is all backed by comprehensive documentation, including API schemas and deployment guides, vital for team collaboration and future development.

5.2.3.6 Portability

Portability was a key outcome achieved through our infrastructure setup. As mentioned in Section 5.2.2.7, the application is containerized and can run consistently across various platforms and environments with no modification required. This ensures our system remains easy to deploy and

operate regardless of the underlying infrastructure, whether in local development, staging, or production. Configuration via environment variables and dependency isolation further supports seamless transferability. While current reliance on external services introduces some limitations in installation simplicity, these will be addressed in future updates. See Figure 5.2.2.7.1 for the architecture overview.

5.2.4 Sample Source Code

The sample source codes are documented in the Appendices (Figure 5.2.4.1 to Figure 5.2.4.4), each showcases a core component of the system:

- Figure 5.2.4.1 presents the function responsible for retrieving and parsing patent documents from USPTO sources into structured sections, including title, abstract, claims, inventors, and metadata.
- Figure 5.2.4.2 showcases the orchestration of the full classification pipeline, integrating problem extraction, guardrail validation, context retrieval, and TRIZ multi-label classification.
- Figure 5.2.4.3 details the functions used to manage Qdrant vector collections, supporting hybrid search through dense and sparse embeddings for effective context retrieval.
- Figure 5.2.4.4 outlines the primary module for handling REST API requests, which coordinates document processing workflows triggered by file uploads and chatbot queries.

6.0 Critical Discussion

6.1 Project Overall Success

In our initial project proposal (FIT3163), we aimed to create an intelligent application that leverages a Large Language Model (LLM) to automate the classification of eco-solution patents by the 40 TRIZ Inventive Principles. While we successfully delivered an innovative proof-of-concept, the EcoPatent Analyzer, its realised outcome evolved significantly from our initial vision. The project succeeded in its technical ambition to build a novel, end-to-end system; however, its ultimate value and success are best measured not by the quantitative classification metrics we first

set out to optimize, but by the qualitative strength and explainability of the tool as an intelligent assistant.

Our initial focus was on the quantitative performance of the multi-label classification task. Prioritizing the equal importance of each of the 40 TRIZ principles, we used label-based aggregation metrics for evaluation. The performance against our automatically generated ground truth was as follows:

- Macro-Averaged Precision: 0.1450
- Macro-Averaged Recall: 0.2602
- Macro-Averaged F1 Score: 0.1459

These low scores reflect the immense difficulty of the task, a key finding that reshaped our understanding of the project. Our analysis identified two primary causes: high data sparsity within the patent dataset and, more critically, potential overfitting from our automated ground truth generation method. This process, which relied on high-dimensional embeddings to interpret patent claims, likely introduced noise that caused the model to fit to spurious patterns rather than generalizable inventive principles. This insight is a critical outcome in itself: creating a reliable evaluation benchmark for such an abstract task is a major challenge that cannot be overcome without extensive, manual annotation by domain experts.

Consequently, our thinking changed during execution. We recognized that the project's definitive success lay in the qualitative value of its core features. The most significant achievement became the implementation of our novel "reasoning trace injection" method, which dramatically improved the consistency and explainability of the LLM's outputs. This pivots the tool's value proposition from being a high-precision, autonomous classifier to an intelligent assistant. Its primary purpose is to empower a user by surfacing potential patent-to-principles mapping and providing the underlying reasoning for each suggestion. In this, the tool excels at helping users generate initial hypotheses and explore novel connections, fulfilling our project vision.

We fully acknowledge the project's primary limitation: the lack of a TRIZ domain expert to validate the nuanced classifications. While the system is functionally robust and its explainability is qualitatively strong, collaboration with a subject matter expert is the necessary next step to validate the model's practical utility formally. Ultimately, this experience demonstrates that a more meaningful quantitative analysis would require a complete shift in evaluation methodology, focusing on the quality of retrieval and reasoning rather than simple classification accuracy, a point which will be expanded upon in the conclusion. Therefore, the EcoPatent Analyzer project was a success not just in building the foundational architecture, but in demonstrating the viability of an explainability-focused approach and clearly defining the path forward for future, expert-guided refinement.

6.2 Project Management

The management of our project required a flexible yet structured approach to handle the complexities of integrating advanced AI technologies. Our strategies evolved from initial planning through to execution, responding to various challenges and learning opportunities.

6.2.1 Methodology

- **Proposed:** We planned to adopt an Agile process model to support iterative development, allowing us to incorporate refinements and user feedback throughout the project's lifecycle.
- **Reflection on Actual Implementation:** Our approach was best described as an informal but effective iterative process. While we did not conduct formal Agile sprints, we maintained its core principles through weekly review cycles and continuous integration. This organic adaptation of Agile allowed us to remain flexible and respond dynamically to unforeseen technical hurdles, underscoring the value of adaptability when working with unpredictable emerging technologies.

6.2.2 Schedule and Scope

- **Proposed:** The project timeline was detailed in a Gantt Chart and Work Breakdown Structure (WBS). The scope was defined to include multi-document analysis, similarity scoring, and a persistent database for user uploads.
- **Reflection on Actual Schedule and Scope:** The project schedule became highly adaptive. The primary cause for deviation was the unreliability of our Selenium-based web scraper, a direct manifestation of the 'Data Source Instability' risk that we had underestimated during our initial planning (Figure 5.1.3). This forced us to de-scope major features, most notably the persistent database and multi-document analysis. This decision was a crucial risk-management strategy; it allowed us to redirect our resources toward perfecting the core single-document analysis pipeline, ensuring a high-quality, functional deliverable rather than a broader but unstable one.

6.2.3 Team Management and Collaboration

- **Proposed:** We outlined specific roles and responsibilities for each team member to ensure a coordinated development process.
- **Reflection on Actual Team Management:** Our team management practices were characterized by a blend of structured planning and organic adaptation. For Task Management, we maintained a live Gantt chart, which was reviewed in weekly meetings to track progress against milestones and manage dependencies. Our Communication relied on WhatsApp for real-time coordination, while weekly meetings with our supervisor were essential for strategic guidance and unblocking challenges. For Version Control, we used GitHub for all code, employing a feature-branching workflow and pull requests for peer

review, which maintained code quality and prevented integration conflicts. Finally, Role Allocation evolved organically from our initial plan; over time, clear ownership emerged based on individual strengths, with members focusing on frontend, backend, and research, allowing us to work effectively and in parallel.

6.2.4 Risk Management

- **Proposed:** We planned to use a risk register to identify and mitigate potential project risks.
- **Reflection on Actual Risk Management:** Our proactive risk management was pivotal. However, we underestimated the impact of two key risks. First, Data Source Instability proved to be a persistent challenge that consumed significant development time and directly led to de-scoping major features. Our response was reactive (repeatedly fixing the scraper) rather than pre-emptive. A more robust initial risk assessment would have involved testing the data source's stability before finalizing a scope that depended on it. In our initial risk register (Figure 5.1.2), we identified 'Dataset Loss' (R2) but underestimated the more immediate risk of 'Data Source Instability.' Our initial focus was on data preservation rather than the reliability of the collection pipeline itself. This was a key lesson in risk assessment.

  Second, we navigated the risks associated with Managing LLM Constraints. Using a powerful, closed-source model like GPT-4o introduces a trade-off between performance and practicality. The high latency and API costs associated with our complex, multi-stage prompts were significant technical and financial risks. We mitigated this issue by optimizing prompts and caching results where possible, but the experience highlighted a critical lesson: effective LLM implementation requires striking a balance between model capability, performance, and cost.

The EcoPatent Analyzer project was a definitive success. It achieved its primary goal of creating a functional and intelligent tool that empowers users to analyze complex patents using the TRIZ framework. We delivered a robust proof-of-concept that is both technically innovative and practically useful. While we de-scoped certain ancillary features due to pragmatic technical constraints, the core classification and chatbot functionalities exceeded the quality envisioned in the original proposal, particularly in terms of their explainability and sophistication. The journey from proposal to final product demonstrates successful, adaptive project management, where facing and overcoming unforeseen challenges led to a more focused and ultimately more intelligent final system.

7.0 Conclusion

In summary, this project has successfully met its primary goals by developing the EcoPatent Analyzer, a foundational system designed to classify patents using the TRIZ methodology. We

began by establishing the context of eco-innovation and the challenges of manual patent analysis, justifying the use of LLMs and a sophisticated Retrieval-Augmented Generation (RAG) pipeline. The core of our work, detailed in the methodology, involved implementing a multi-label classification system using multi-stage prompting with GPT-4o and developing an interactive chatbot powered by a hybrid retrieval system combining BM42 and Ada-002 embeddings. The key achievement was not the creation of a functionally robust system, but the successful demonstration of an explainability-focused approach, which was central to the project's vision.

The resulting Patent Analyzer stands as a successful proof-of-concept for applying explainable AI to the nuanced domain of patent analysis. Our evaluation yielded mixed, yet insightful, results. While the quantitative metrics for the TRIZ classification were low, highlighting the immense difficulty of automated labeling without expert validation, our RAGAS evaluation confirmed the chatbot's ability in providing factually grounded, context-aware responses.

The success of this project is evaluated on two fronts. Firstly, the delivery of the foundational architecture itself, which performs reliably. Secondly, and more significantly, the project successfully demonstrated the viability of this explainable approach and clearly identified the necessary direction for future work. We acknowledge the limitation of this project: the lack of a domain expert to validate the nuanced classifications made by the model formally. This insight has been critical in shaping our evaluation of success. We have concluded that meaningful quantitative validation requires a paradigm shift from measuring simple classification accuracy to evaluating the quality of the tool's information retrieval and reasoning.

To conclude, the EcoPatent Analyzer has successfully achieved its core objective of delivering a working, explainable system for patent analysis. By laying this essential groundwork and clearly defining the path toward expert-guided refinement, this project serves as a critical first step. Its true impact lies not only in the tool itself, but in providing a blueprint for applying explainable AI to other nuanced, high-stakes domains, ultimately fostering a more transparent and impactful approach to innovation research.

8.0 Appendices

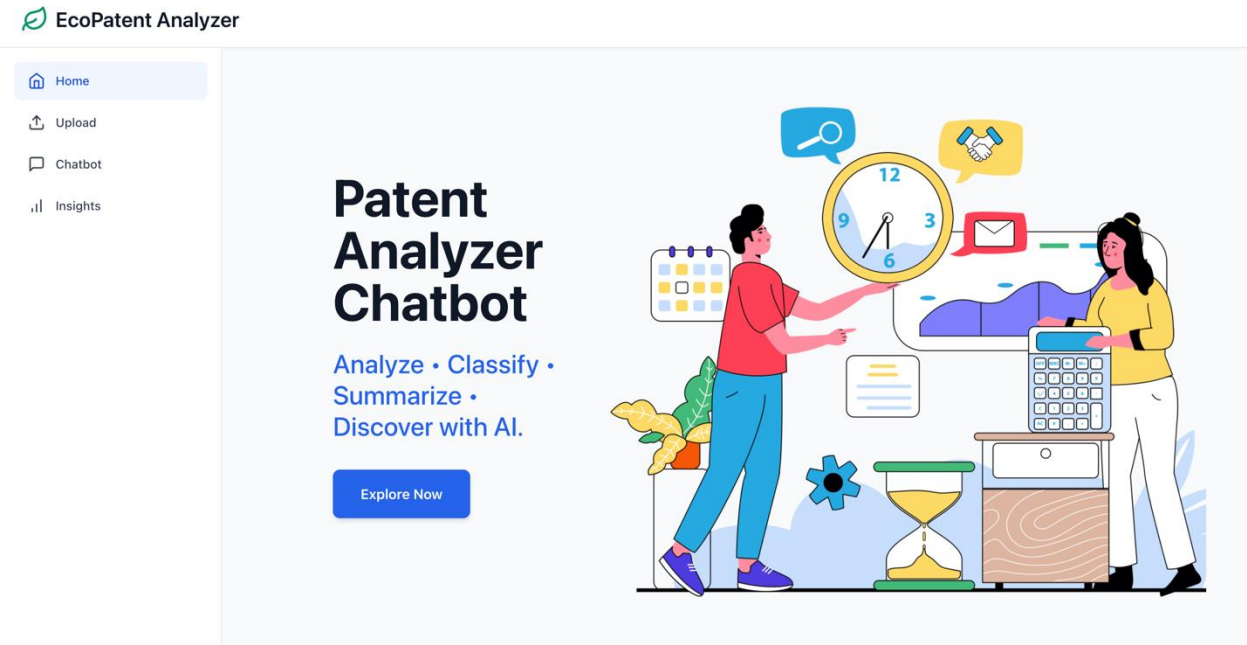


Figure 3.1.1: Hero Section (Home Page)

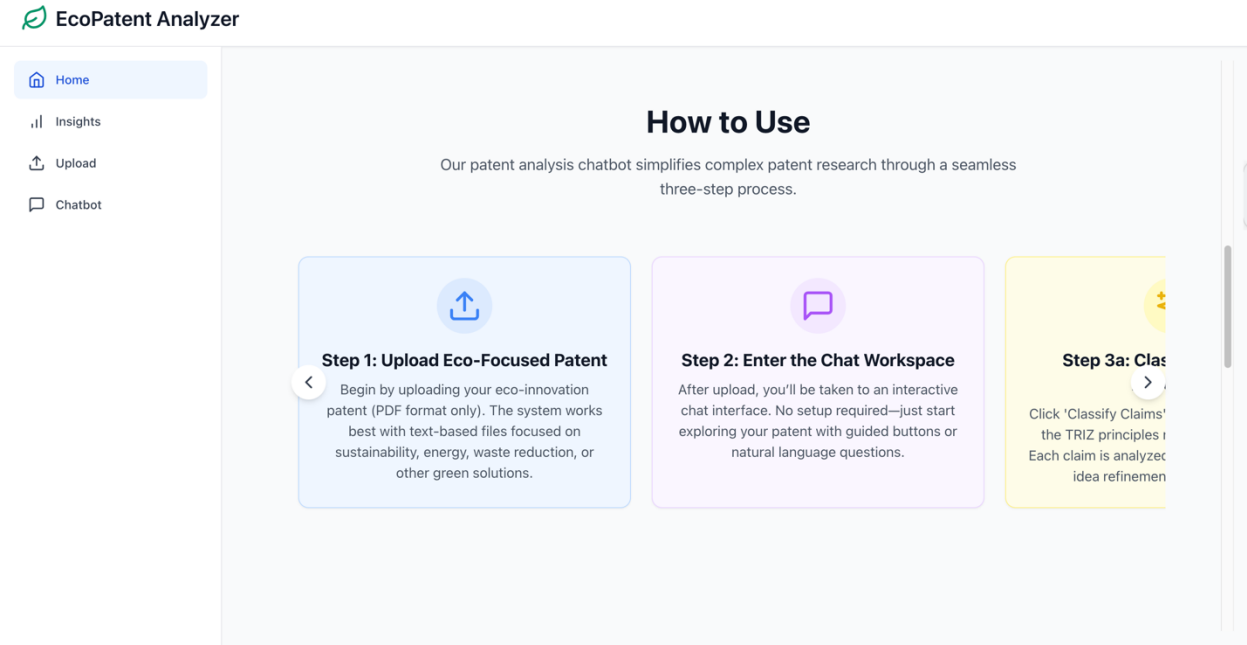


Figure 3.1.2: How It Works (Home Page)

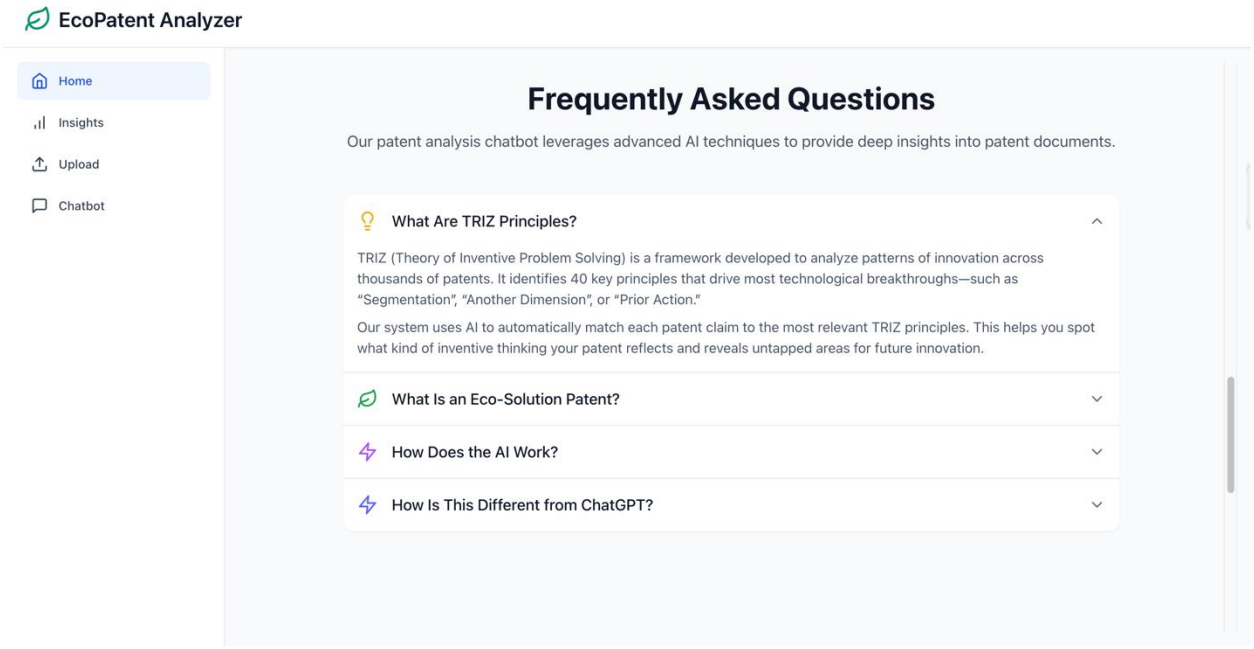


Figure 3.1.3: Frequently Asked Questions (Home Page)

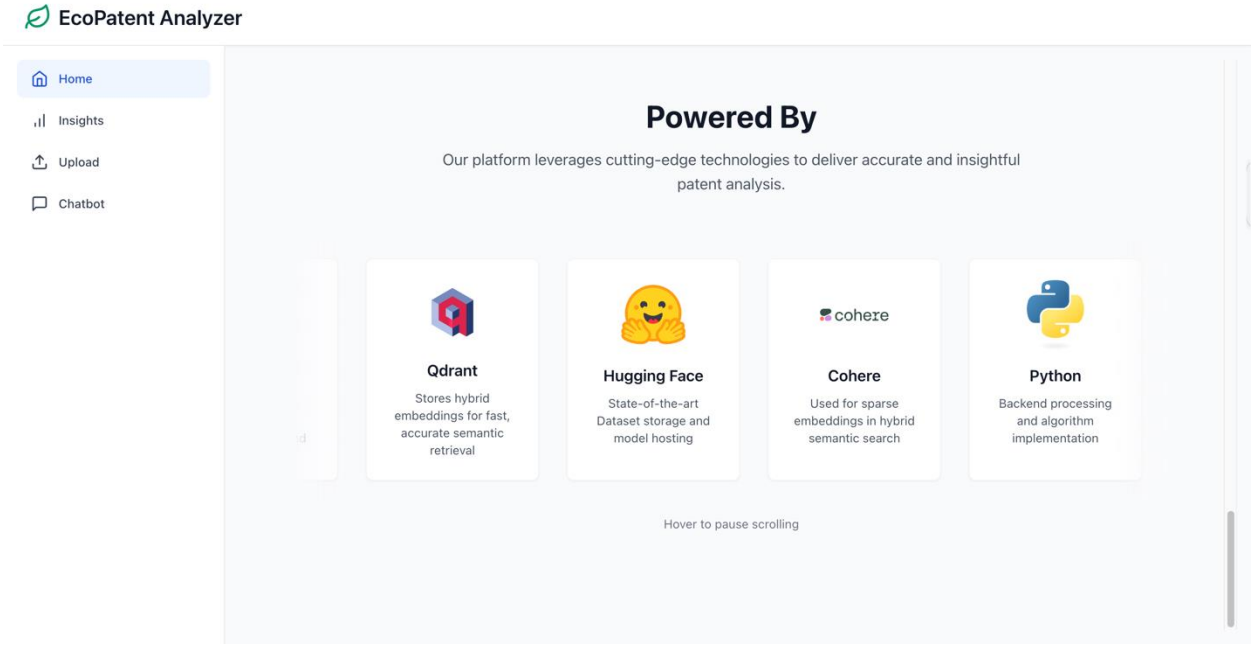


Figure 3.1.4: Technology Stack (Home Page)

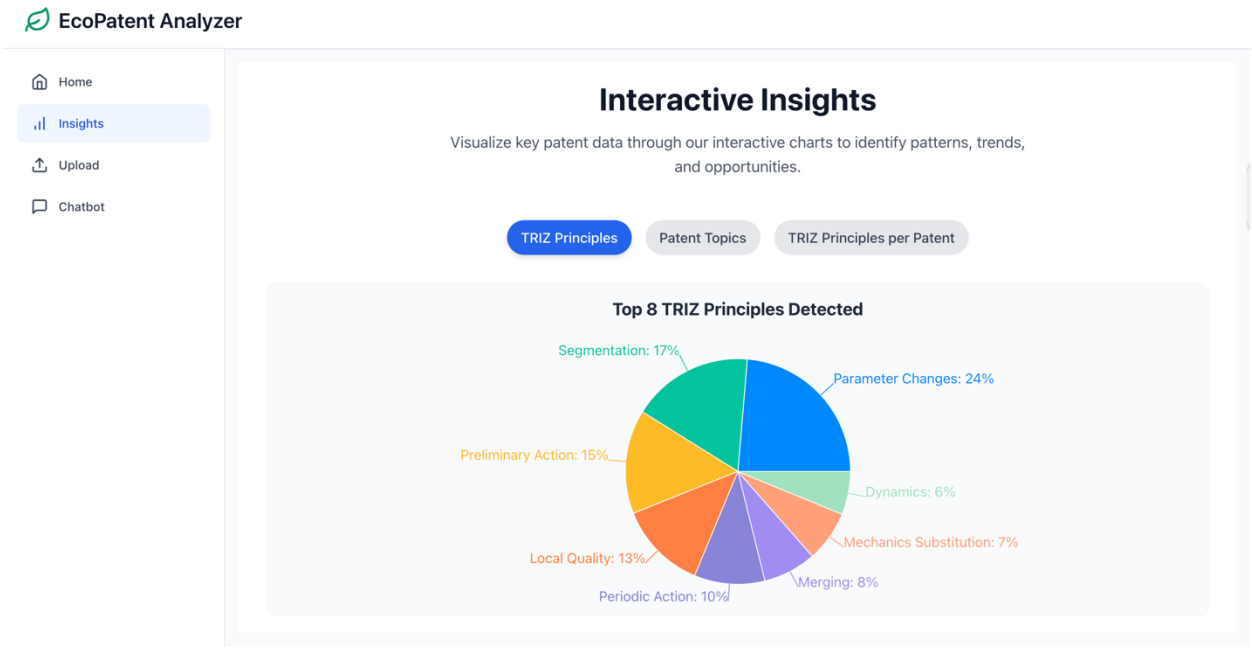


Figure 3.1.5: Insights Page (1)

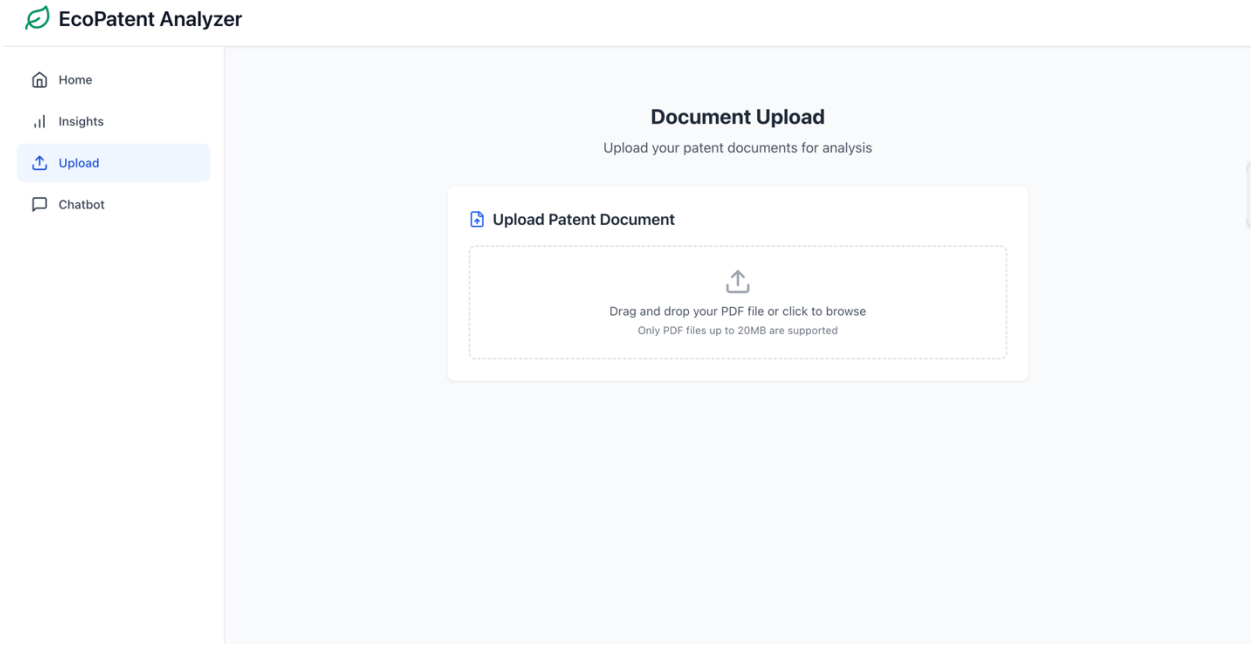


Figure 3.1.6: Upload Page

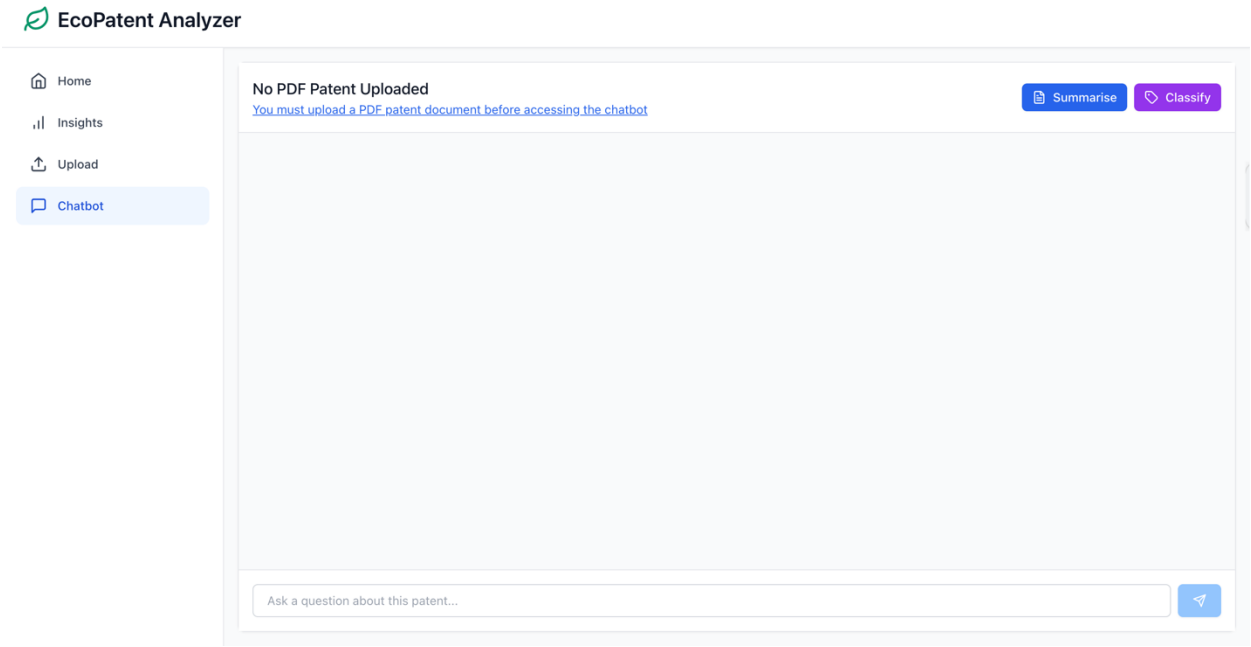


Figure 3.1.7: Chatbot Page

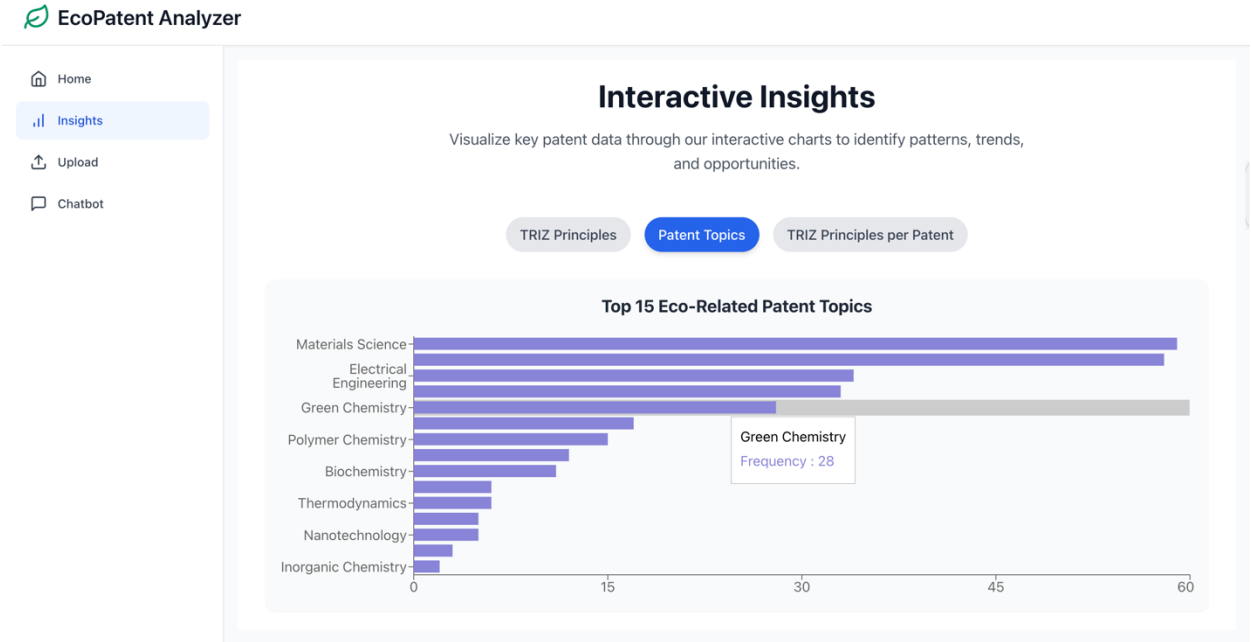


Figure 5.2.2.5.1: Insights Page (2)

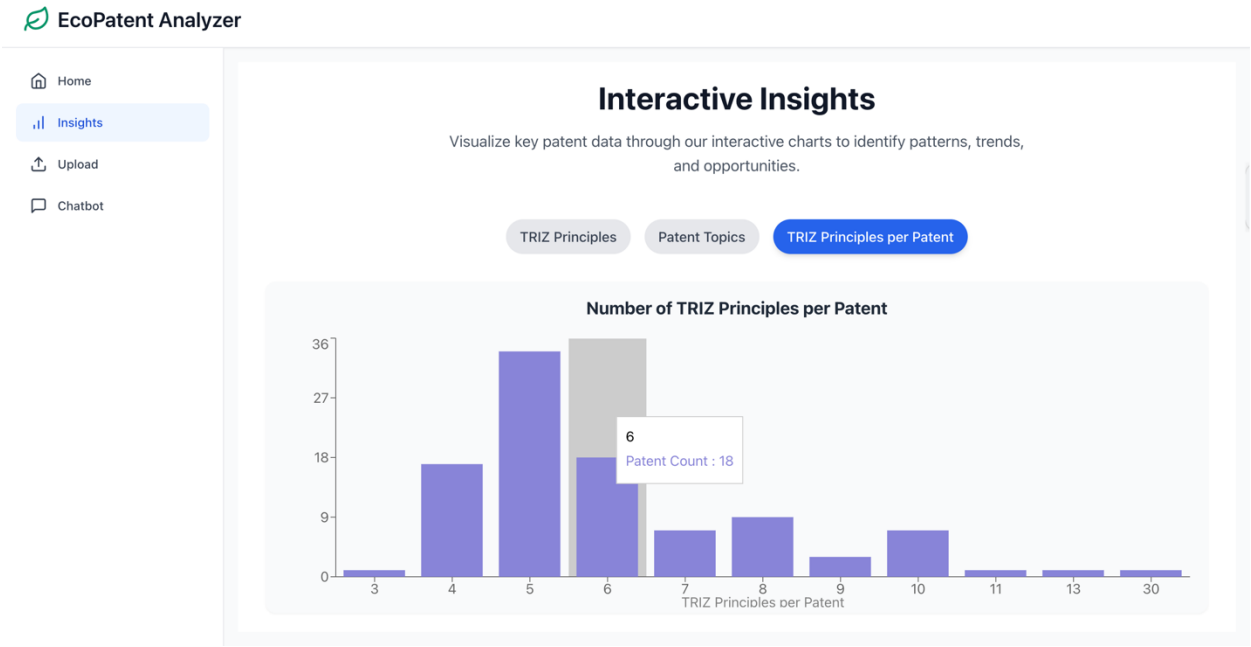


Figure 5.2.2.5.2: Insights Page (3)

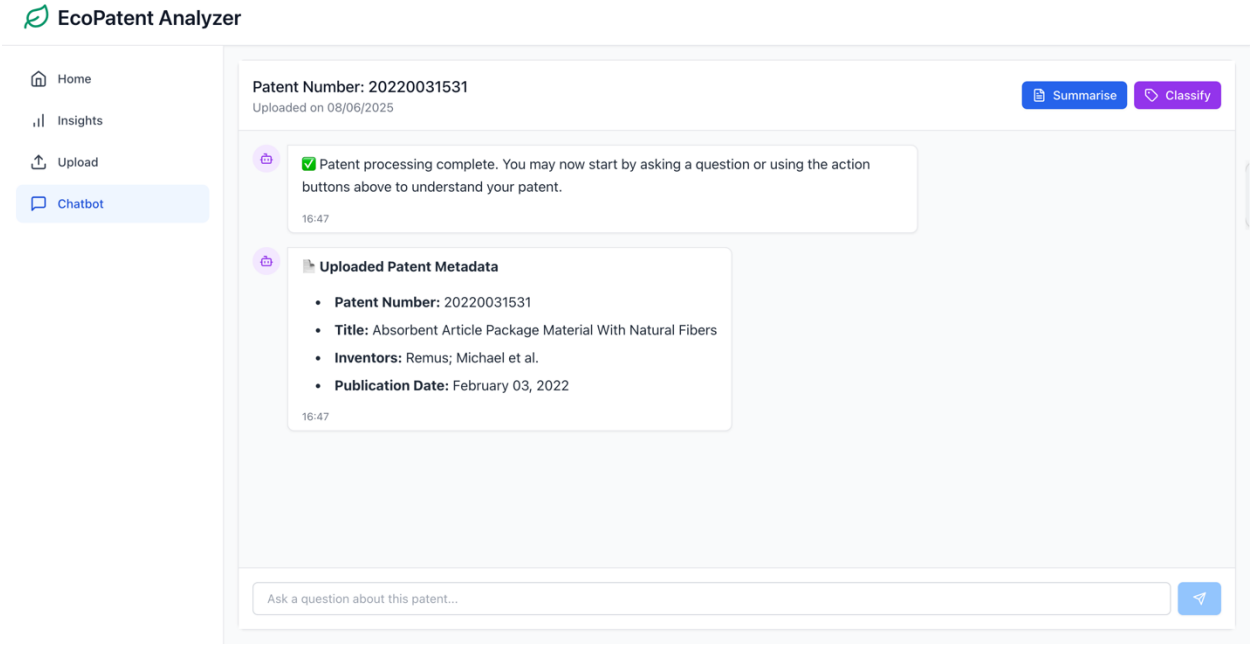
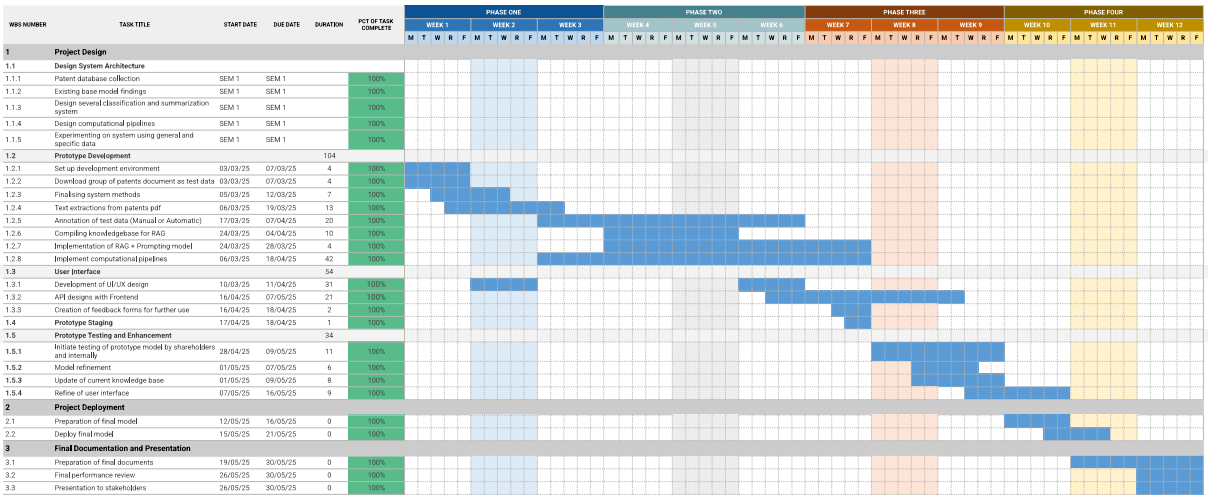


Figure 5.2.2.5.4: Patent Upload Successful (Chatbot Page)

GANTT CHART MDS08



WBS NUMBER	TASK TITLE	START DATE	DUE DATE	DURATION	PCT OF TASK COMPLETE
1	**Project Design**				
1.1	**Design System Architecture**				
1.1.1	Patent database collection	SEM 1	SEM 1		100%
1.1.2	Existing base model findings	SEM 1	SEM 1		100%
1.1.3	Design several classification and summarization system	SEM 1	SEM 1		100%
1.1.4	Design computational pipelines	SEM 1	SEM 1		100%
1.1.5	Experimenting on system using general and specific data	SEM 1	SEM 1		100%
1.2	**Prototype Development**			104	
1.2.1	Set up development environment	03/03/25	07/03/25	4	100%
1.2.2	Download group of patents document as test data	03/03/25	07/03/25	4	100%
1.2.3	Finalising system methods	05/03/25	12/03/25	7	100%
1.2.4	Text extractions from patents pdf	06/03/25	19/03/25	13	100%
1.2.5	Annotation of test data (Manual or Automatic)	17/03/25	07/04/25	20	100%
1.2.6	Compiling knowledgebase for RAG	24/03/25	04/04/25	10	100%
1.2.7	Implementation of RAG + Prompting model	24/03/25	28/03/25	4	100%
1.2.8	Implement computational pipelines	06/03/25	18/04/25	42	100%
1.3	**User Interface**			54	
1.3.1	Development of UI/UX design	10/03/25	11/04/25	31	100%
1.3.2	API designs with Frontend	16/04/25	07/05/25	21	100%
1.3.3	Creation of feedback forms for further use	16/04/25	18/04/25	2	100%
1.4	**Prototype Staging**	17/04/25	18/04/25	1	100%
1.5	**Prototype Testing and Enhancement**			34	
1.5.1	Initiate testing of prototype model by shareholders and internally	28/04/25	09/05/25	11	100%
1.5.2	Model refinement	01/05/25	07/05/25	6	100%
1.5.3	Update of current knowledge base	01/05/25	09/05/25	8	100%
1.5.4	Refine of user interface	07/05/25	16/05/25	9	100%
2	**Project Deployment**				
2.1	Preparation of final model	12/05/25	16/05/25	0	100%
2.2	Deploy final model	15/05/25	21/05/25	0	100%
3	**Final Documentation and Presentation**				
3.1	Preparation of final documents	19/05/25	30/05/25	0	100%
3.2	Final performance review	26/05/25	30/05/25	0	100%
3.3	Presentation to stakeholders	26/05/25	30/05/25	0	100%

Figure 5.1.1: Gantt Chart

For a clearer view, please refer to the following link: [MDS08 Gantt Chart]

Risk Register

Prepared by: MDS08

No.	Rank	Risk	Description	Category	Triggers	Root Cause	Potential Responses	Risk Owner	Probability	Impact	Status	Score (Probability*Impact)
R1	2	Member Misses Deadlines	Missed milestones causing delays in dependent tasks	People	Mischecked Gantt chart	Lack of time management	Teammates send reminders, monitor progress closely	Renee	50	4	Monitored	200
R2	3	Dataset Loss	Processed dataset is lost or becomes inaccessible	People, Software	Data loss in shared folder, lack of backup	Shared folder/ system failure, careless data management	Implement backup database, assign a person to manage data input	Steve	30	6	Monitored	180
R3	1	Insufficient Computation Power	Lack of computation power to encode and decode dataset and model	Software	Email from cloud provider regarding tier limitations	Free tier inadequate for computational needs	Request funds for a higher cloud service tier	Steve	70	7	Monitored	490
R4	5	Redundant Work	Lack of communication and understanding of each role causing twice the amount of work.	People	A conflict in our log entry and gant chart, who is responsible to what work. during team update	Miscommunication about task ownership	Keep Gantt chart updated, improve task allocation, increase meetings	Renee	10	3	Monitored	30
R5	6	Security and Data Privacy Risks	Patent data, especially sensitive information, are not handled securely to avoid data leaks or breaches.	People	Non-compliance with privacy regulations and reputational damage.	Lack of adequate caution in data handling	All team members sign a NDA and committ to data security	Naveed	5	4	Monitored	20
R6	4	Integration Issues	Smooth integration between the frontend, backend APIs, databases, and LLM models can be complex.	Technology	Incompatibility between components could lead to system breakdowns or feature malfunctions.	Incompatible technology selection or inadequate design factors	Review design and development process regorously	Naveed	10	6	Monitored	60
R7	7	Unreachable Supervisor	Delay in receiving feedback or guidance	People	Unavailability of supervisor	Limited communication options	Schedule alternate meetings, document questions for efficient follow-up	Shrinithi	5	2	Monitored	10

Figure 5.1.2: Risk Register

For a clearer view, please refer to the following link: [MDS08 Risk Register]

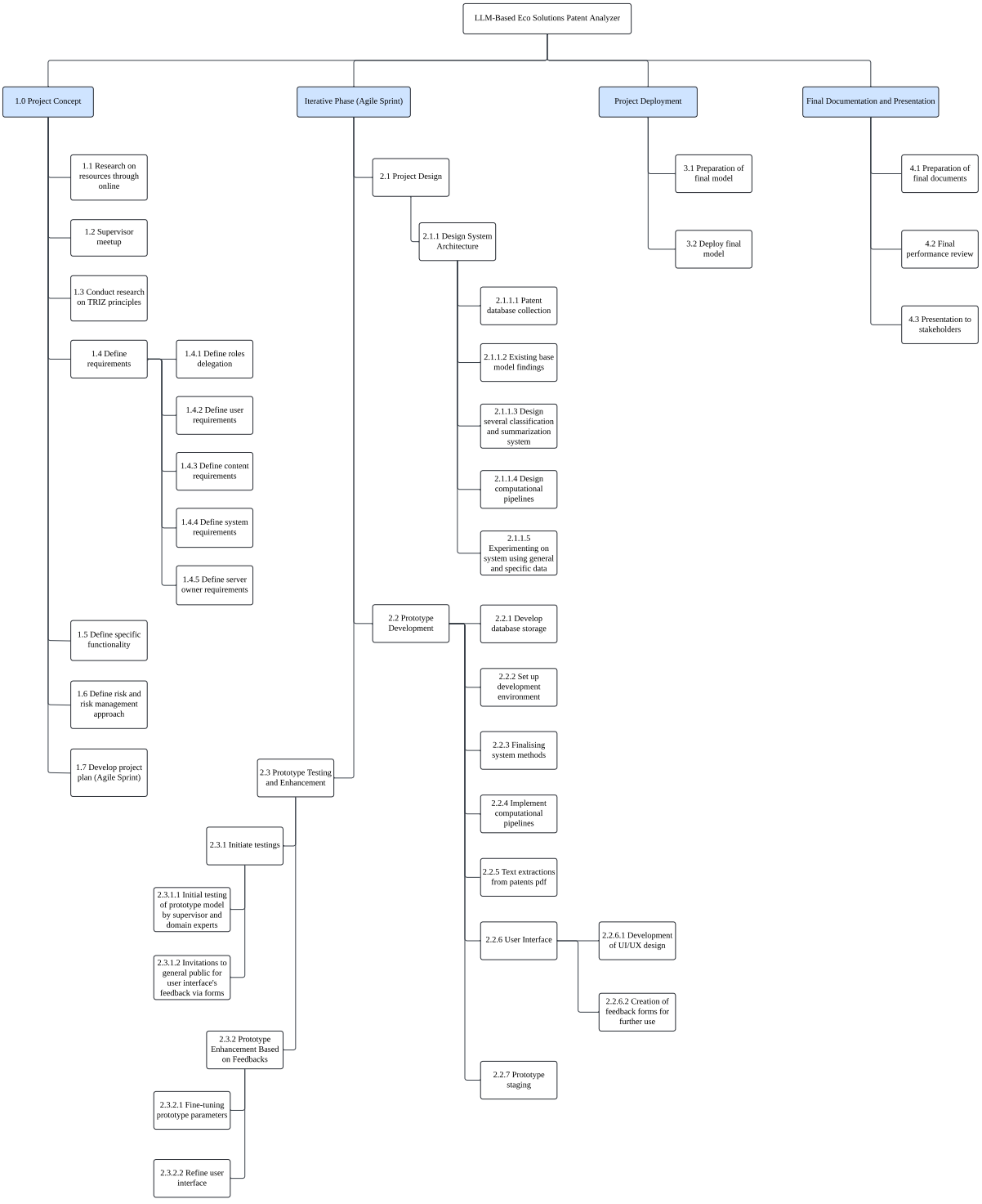


Figure 5.1.3: Work Breakdown Structure (WBS)

For a clearer view, please refer to the following link: [Work Breakdown Structure (WBS)]

Requirement Traceability Matrix (RTM)					
MDS08					
ID	Description	Category	Type	Source	Status
01	Acquire patent data from open-source databases such as USPTO	Data Acquisition	Functional	Dr Ong Huey Fang	Completed
02	Preprocess patent PDF texts using tokenization, vectorization, and structured data cleaning	Data Preparation	Functional	Dr Ong Huey Fang	Completed
03	Ensure the system can scale to handle large volumes of patent documents within the retrieval-augmented generation (RAG) framework without performance degradation.	System Architecture & Scalability	Non-Functional	Dr Ong Huey Fang	Completed
04	Extract key information from patents to support summarization and TRIZ classification using the model	Information Extraction and Model Training	Functional	Dr Ong Huey Fang	Completed
05	Classify patent documents into one or more of the 40 TRIZ inventive principles	Model Classification	Functional	Dr Ong Huey Fang	Completed
06	Benchmark performance using evaluation metrics such as accuracy, latency, and classification confidence	Model Evaluation	Non-Functional	Dr Ong Huey Fang	Completed
08	Provide users with a web interface to upload patents and receive classification and summary outputs	Web Application Development	Functional	Dr Ong Huey Fang	Completed
09	Enable interactive querying of processed patents through a chatbot interface	Conversational Interface Design	Funcational	Dr Ong Huey Fang	Completed
10	Provide responsive and accessible user interface for non-technical users	Usability Engineering	Non-Functional	Dr Ong Huey Fang	Completed
11	Ensure system uptime and backend stability during operation	System Reliability & Infrastructure	Non-Functional	Dr Ong Huey Fang	Completed

Table 3.3.1: Requirement Traceability Matrix (RTM)

User Acceptance Criteria	
MDS08	
User Story	**Acceptance Criteria**
As a developer, I want my model to generate consistent classifications and concise, complete summaries.	The model consistently produces accurate classifications and informative summaries, validated against labeled benchmark datasets.
As a developer, I want my data processing pipeline to be able to reject invalid or incorrectly formatted files.	The system accepts only valid patent PDF documents. Invalid files are rejected, with verification logs recorded in the PostgreSQL database.
As a developer, I want extraction to work consistently, metadata and main texts can be acquired with minimal typo or error.	Extracted data must populate all required metadata fields or contain minimal text errors. Completeness and accuracy are verified through structured PostgreSQL queries.
As a developer, I want patent data to be stored and retrieved efficiently from our database.	Data storage and retrieval performance is validated through system and stress testing, measuring query response time and throughput under various conditions.
As a user, I want an intuitive interface to upload documents, view summaries, and visualize TRIZ mappings clearly.	Users can upload patents and view summaries with clearly highlighted TRIZ mappings. Interface usability is confirmed through testing and survey feedback.
As a user, I want the system to respond quickly and reliably when I query about the uploaded patent data.	The system processes inputs promptly without failure, as confirmed by stress testing and performance benchmarks.
As a user, I want the summaries to be clear, detailed, and useful for understanding without reading the full document.	Summaries are evaluated using expert feedback and benchmark metrics (e.g., ROUGE, coherence) to ensure clarity and relevance.

Table 3.3.2: User Acceptance Criteria

Combined Re-ranking Score Calculation

Definitions:

- Q: Query string (concatenation of topics).
- D_i: Text of the i-th search result.
- $\text{RerankScore}(Q, D_i)$: Relevance score from the Cohere rerank model for (Q, D_i).
- $\text{OriginalScore}(R_i)$: Initial ranking score of the i-th result.
- N_i: Min-max normalized $\text{OriginalScore}(R_i)$.
- X: List of all original scores $\{\text{OriginalScore}(R_1), \ldots, \text{OriginalScore}(R_n)\}$.

Formulas:

Min-max Normalization of Original Scores:

$$N_j = \frac{\text{OriginalScore}(R_j) - \min(X)}{\max(X) - \min(X)}$$

Combined Re-ranking Score (Weighted Average):

$$C_i = 0.7 \cdot \text{RerankScore}(Q, D_i) + 0.3 \cdot N_i$$

Results are sorted in descending order of C_i to produce the final ranked list.

Figure 4.3.2 Reranked Adjustment Formula

```python
    def process_document(self, filename: str) -> PatentDocument:
        """
        Main processing pipeline for a patent document.

        Args:
            filename: USPTO document identifier

        Returns:
            Populated PatentDocument instance
        """
        html = self.retrieveHTML(filename)

        if not html:
            raise ValueError("Failed to retrieve HTML content.")

        soup = BeautifulSoup(html, 'html.parser')

        # Extract metadata
        title_element = soup.find('h2', class_='bottom-border padding')
        if title_element:
            self.document.title = title_element.get_text(strip=True)

        inventor_label = soup.find('span', string="Inventor(s)")
        if inventor_label:
            self.document.inventor = inventor_label.find_next('span').get_text(strip=True)

        date_label = soup.find('span', string="Publication Date")
        if date_label:
            self.document.publication_date = date_label.find_next('span').get_text(strip=True)

        # Extract sections
        self.document.abstract = self.extract_section_text(soup, 'Abstract')
        self.document.background_summary = self.extract_section_text(soup,
'Background/Summary')
        self.document.description = self.extract_section_text(soup, 'Description')

        # Claims require special handling
        claims_header = soup.find('h3', string='Claims')
        if claims_header:
            claims_section = claims_header.find_parent('section')
            self.document.claims = claims_section.get_text(separator=' ', strip=True)
        return self.document
```

Figure 5.2.4.1: Document processing pipeline

```python
def classify_patent(self, abstract: str, claims: str) -> TRIZPrinciple:
        """
        Classifies a patent based on its abstract and claims.

        Args:
            abstract (str): The patent abstract
            claims (str): The patent claims

        Returns:
            TRIZPrinciple: The classification result

        Raises:
            ValueError: If the problem extraction or classification fails
            Exception: For other processing errors
        """
        try:
            # Extract problems
            extraction_prompt = Prompt.PROBLEM_EXTRACTION.value.format(claims=claims)
            problems_raw = self.model.chat(extraction_prompt, 3)

            # Apply guardrail
            if not self.add_guardrails(problems_raw):
                raise ValueError("Problem extraction failed guardrail validation")

            # Parse problems dictionary
            try:
                problems_dict = ast.literal_eval(problems_raw.strip("```python\n").strip("\n```"))
            except (SyntaxError, ValueError) as e:
                raise ValueError(f"Failed to parse problems dictionary: {str(e)}")

            # Extract topics
            topic_prompt = Prompt.TOPIC_PROMPT.value.format(abstract=abstract)
            try:
                topic_list = ast.literal_eval(self.model.chat(topic_prompt,
0).strip("```python\n").strip("\n```"))
            except (SyntaxError, ValueError) as e:
                raise ValueError(f"Failed to parse topic list: {str(e)}")

            # Retrieve context and perform analysis
            context = self.retrieve_context(20, "allenAI_chemData", topic_list, problems_dict)
            analysis_prompt = Prompt.PROBLEM_ANALYSIS.value
            analysis = self.model.chat(analysis_prompt.format(problems=problems_dict,
reasoning_trace=context), 5)

            # Generate dynamic rule
            rule_prompt = Prompt.RULE_CREATION.value
            dynamic_rule = self.model.chat(rule_prompt.format(analysis=analysis), 2)

            # Final classification
            final_prompt = Prompt.FINAL_CLASSIFICATION.value
            raw = self.model.chat(final_prompt.format(dynamic_rule=dynamic_rule, claims=claims), 0)

            return ClassificationPipelineOutput(
                extracted_problems=problems_dict,
                topics=topic_list,
                context=context,
                analysis=analysis,
                dynamic_rule=dynamic_rule,
                final_classification=raw
            )

        except ValueError as ve:
            print(f"Validation error in patent classification: {str(ve)}")
            raise
        except Exception as e:
            print(f"Error in patent classification: {str(e)}")
            raise
```

Figure 5.2.4.2: Multi-label TRIZ classification engine

```
    def retrieve_chunks(self, query, top_k=10):
        # Get embeddings for the query
        dense_embedding = self.openai_client.embed(query)
        sparse_embedding = list(self.model_bm42.query_embed(query))[0]

        # Perform hybrid search using prefetch and fusion
        results = self.qdrant.query_points(
            collection_name=self.collection_name,
            prefetch=[
                models.Prefetch(query=sparse_embedding.as_object(), using="sparse",
limit=top_k),   models.Prefetch(query=dense_embedding, using="dense", limit=top_k),
            ],
            query=models.FusionQuery(fusion=models.Fusion.RRF),
                search_params=models.SearchParams(
            quantization=models.QuantizationSearchParams(
                ignore=False,
                rescore=True,
                oversampling=2.0,
            )),
            limit=top_k
        )

        # Simply append all results
        return " ".join(point.payload['text'] for point in results.points)

    def generate_answer(self, query, context):
        history = self.memory.load_memory_variables({}).get("chat_history", "")
        prompt = f"""
Context:
{context}

History:
{history}

User Question: {query}

Please provide a clear and concise answer based on the context above.
"""
        return self.openai_client.chat(prompt, 0.7)

    def init_chatbot(self, text: PatentDocument):
        full_text = str(text)
        points = self.embed_chunks(full_text)
        self.create_qdrant_index(points)


    def generate_patent_answer(self, query: str):

        # Retrieve relevant chunks for the query
        retrieved = self.retrieve_chunks(query)

        # Build context from reranked chunks
        context = "".join(retrieved)

        # Generate final answer from context
        answer = self.generate_answer(query, context)
        print(context)

        # Save memory for the next interactions
        self.memory.save_context({"query": query}, {"output": answer})

        return answer
```

Figure 5.2.4.3: Vector database integration and query processing

```python
app = Flask(__name__)
CORS(app)

# --- New Improved Regex Patterns ---
PATENT_NUM_PATTERN = re.compile(
    rb'(?:US[-\s]*)(\d{5,11})(?:[-\s]*A1)?',
    re.IGNORECASE
)
TEXT_PATTERN = re.compile(
    r'(?:US[-\s]*)(\d{5,11})(?:[-\s]*A1)?',
    re.IGNORECASE
)
# --- Service Initializations ---
processor = DocProcessing()
classifier = PatentClassifier(Openai("OPENAI_KEY"))
globalPatent = None
chatbot = None
summarization = None

@app.route('/upload', methods=['POST'])
def upload_pdf():
    global globalPatent
    if 'file' not in request.files:
        return jsonify({'error': 'No file part'}), 400

    file = request.files['file']

        # Process and classify
        try:
            patent_doc = processor.process_document(extracted_num)
            print(f"DEBUG: Processed patent document: {patent_doc.title}")
            raw_result = classifier.classify_patent(patent_doc.abstract, patent_doc.claims)
            summarization = classifier.summarization(patent_doc)
            suggested_questions =
classifier.generate_suggested_questions(summarization+raw_result.final_classification)
            globalPatent = patent_doc

            return jsonify({
                'message': 'PDF processed and classified successfully',
                'patent_number': extracted_num,
                'title': patent_doc.title,
                'Inventors': patent_doc.inventor,
                'publication_date': patent_doc.publication_date,
                'classification_result': raw_result.final_classification,
                'summ': summarization,
                'suggested_questions': suggested_questions
            }), 200

        except ValidationError as ve:
            return jsonify({'error': 'Validation failed', 'details': ve.errors()}), 422
        except Exception as e:
            print(f"[Processing Error]: {str(e)}")
            return jsonify({'error': f'Error during processing/classification: {str(e)}'}), 500

    except Exception as e:
        print(f"[Upload Error]: {str(e)}")
        return jsonify({'error': f'Failed to process PDF: {str(e)}'}), 500

@app.route('/query', methods=['POST'])
def interact_query():
    global chatbot, globalPatent
    if not globalPatent:
        return jsonify({'error': 'No document uploaded or processed. Please upload a PDF first.'}), 400

    if not chatbot:
        chatbot = PatentChatbot(globalPatent)

    data = request.get_json()
    if not data or 'question' not in data:
        return jsonify({'error': 'No question provided'}), 400

    question = data['question'].strip()
    try:
        result = chatbot.generate_patent_answer(question)
        return jsonify({'answer': result}), 200
    except KeyError as ke:
        return jsonify({'error': str(ke)}), 500
    except Exception as e:
        return jsonify({'error': f'Error generating response: {str(e)}'}), 500

if __name__ == '__main__':
    app.run(debug=True, host="0.0.0.0", port=8000)
```

Figure 5.2.4.4: RESTful API gateway and service arches

9.0 References

Albino, V., Ardito, L., Dangelico, R. M., & Petruzzelli, A. M. (2014). Understanding the development trends of low-carbon energy technologies: A patent analysis. *Applied Energy*, *135*, 836–854. https://doi.org/10.1016/j.apenergy.2014.08.012

Associados, Á. D. &. (2025, May 6). *Green Patents*. Álvaro Duarte & Associados. https://aduarteassoc.com/green-patents/?lang=en#:~:text=Thus%2C%20innovations%20in%20electric%20vehicles,emissions%20while%20promoting%20cleaner%20transportation.

Cascini, G., & Russo, D. (2006). Computer-aided analysis of patents and search for TRIZ contradictions. *International Journal of Product Development*, *4*(1/2), 52. https://doi.org/10.1504/ijpd.2007.011533

Chiang, W., Zheng, L., Sheng, Y., Angelopoulos, A. N., Li, T., Li, D., Zhang, H., Zhu, B., Jordan, M., Gonzalez, J. E., & Stoica, I. (2024, March 7). *Chatbot Arena: an open platform for evaluating LLMs by human preference*. arXiv.org. https://arxiv.org/abs/2403.04132

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023, September 26). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv.org. https://arxiv.org/abs/2309.15217

EXPLAINER: How Intellectual property rights encourage green Innovation. (n.d.). Ipday. https://www.wipo.int/en/web/ipday/2020/green_future

Faria, L. G. D., & Andersen, M. M. (2014). Understanding the evolution of eco-innovative activity in the automotive sector: a patent based analysis. *ResearchGate*.

https://www.researchgate.net/publication/298790113_Understanding_the_evolution_of_eco-innovative_activity_in_the_automotive_sector_a_patent_based_analysis

Herrera, F., Charte, F., Rivera, A. J., & Del Jesus, M. J. (2016). Multilabel classification. In *Springer eBooks*. https://doi.org/10.1007/978-3-319-41111-8

Johnson, S. (2025b, March 19). *Research spotlight: is long chain-of-thought structure all that matters when it comes to LLM reasoning distillation?* Snorkel AI. https://snorkel.ai/blog/research-spotlight-is-long-chain-of-thought-structure-all-that-matters-when-it-comes-to-llm-reasoning-distillation/

Kamath, U., Keenan, K., Somers, G., & Sorenson, S. (2024). *Large Language Models: a deep dive*. https://doi.org/10.1007/978-3-031-65647-7

Kasliwal, N. (2023, September 18). *Binary Quantization - Vector Search, 40x faster*. Qdrant. https://qdrant.tech/articles/binary-quantization/

IP Business Academy. (2025, April 23). Patent Information in the Steel Industry: A Catalyst for Innovation and Competitiveness - IP Business Academy. *IP Business Academy - Your global IP-Management Education Platform*. https://ipbusinessacademy.org/patent-information-in-the-steel-industry-a-catalyst-for-innovation-and-competitiveness#:~:text=This%20involves%20conducting%20thorough%20patent%20searches%20during,a%20strong%20market%20position%20without%20legal%20challenges.

Loh, H. T., He, C., & Shen, L. (2005). Automatic classification of patent documents for TRIZ users. *World Patent Information*, *28*(1), 6–13. https://doi.org/10.1016/j.wpi.2005.07.007

Russo, D., & Spreafico, C. (2020). TRIZ-Based Guidelines for Eco-Improvement. *Sustainability*, *12*(8), 3412. https://doi.org/10.3390/su12083412

Senghor, O., Ndiaye, M., & Gaye, K. (2023). Unified Eco-Innovation Methodology based on TRIZ and C-K. In *IFIP advances in information and communication technology* (pp. 197–210). https://doi.org/10.1007/978-3-031-42532-5_15

Sojka, V., & Lepšík, P. (2020). Use of TRIZ, and TRIZ with Other Tools for Process Improvement: A Literature Review. *Emerging Science Journal*, *4*(5), 319–335. https://doi.org/10.28991/esj-2020-01234

STEM-AI-MTL/Electrical-Engineering · Datasets at Hugging Face. (2024b, July 23). https://huggingface.co/datasets/STEM-AI-mtl/Electrical-engineering

Tseng, Y., Lin, C., & Lin, Y. (2007). Text mining techniques for patent analysis. *Information Processing & Management*, *43*(5), 1216–1247. https://doi.org/10.1016/j.ipm.2006.11.011

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022, January 28). *Chain-of-Thought prompting elicits reasoning in large language models*. arXiv.org. https://arxiv.org/abs/2201.11903

Vasnetsov, A. (2024, July 1). *BM42: New baseline for hybrid search*. Qdrant. https://qdrant.tech/articles/bm42/?q=bm25

Venugopalan, S. (n.d.). *Evaluating progress of LLMs on scientific problem-solving*. Retrieved April 3, 2025, from https://research.google/blog/evaluating-progress-of-llms-on-scientific-problem-solving/

Yahnoosh. (n.d.). *Hybrid search scoring (RRF) - Azure AI Search*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/search/hybrid-search-ranking

Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., & Hou, L. (2023, November 14). *Instruction-Following evaluation for large language models*. arXiv.org. https://arxiv.org/abs/2311.07911